CSI605: OBJECT-ORIENTED SOFTWARE ENGINEERING

FACULTY OF NATURAL AND APPLIED SCIENCES

SCHOOL OF COMPUTER, MATHEMATICS & STATISTICS SCIENCE



UNIVERSITY PUBLICATION MANAGEMENT SYSTEM

FINAL EVALUATION DOCUMENTATION

Lecturer: Prof Ayalew

GROUP MEMBERS:

1. Keletso Joy Ditlhotlhole-202002023 **(Team Leader)**
2. Kgosi Philip Zilwa-202005452
3. Phazha Chakalisa-202106101
4. Kagiso Mpopang 202607142
5. Archibald Mareme-201904960

Due Date: 06/11/2026

TABLE OF CONTENTS

1. INTRODUCTION **5**

2. JUSTIFICATION OF PROJECT **5**

3. AIM OF THE PROJECT **5**

4. OBJECTIVES **5**

5. REQUIREMENTS SPECIFICATION **6**

Use Case Diagram-System Admin 8

Use Case Diagram-Student 9

Use Case Diagram-Lecturer 10

Use Case Diagram-Reviewer 11

Use Case Diagram-General Public 12

7. USE CASE SPECIFICATIONS **13**

8. INITIAL DOMAIN MODEL **29**

9. REQUIREMENTS PRIORITIZATION **30**

10. DESIGN SPECIFICATION **31**

10.1 Design Goals 31

10.2 Subsystem Responsibilities 32

10.3 Architectural Style Overview 32

Why the Architecture and Subsystems Support our Design Goals 34

Presentation Layer Subsystems 34

Application / Business Logic Layer Subsystems 35

Database / Data Link Layer Subsystem 35

11. ARCHITECTURE **37**

12. Object Domain Model **38**

13. State Chart Diagram **39**

13.1 Publication State Chart 39

14. Activity Diagrams **40**

14.1 Upload Publication Activity Diagram 40

14.2 Review Publication Activity Diagram 41

14.3 Search/Download Activity Diagram 41
15. IMPLEMENTATION AND FILE STRUCTURE 43
15.1 Simplified File Structure 43
15.2 Angular Frontend 44
15.3 Node.js API and Backend 44
15.4 Configuration Files 45
15.5 Controllers 45
15.6 Middleware 45
15.7 Routes 45
15.8 Keycloak Authentication 45
15.9 Notification Service 45
15.10 Database and Supporting Files 46
15.11 Files Excluded from the Documentation 46
15.12 Request Flow 46
16. TEAM RESPONSIBILITIES 47
17. OVERVIEW OF UI 48
18. SYSTEM TESTING 55
18.1 Overview 55
Scope and levels of testing 55
18.2 Test Environment and Test Accounts 55
18.3 Test Cases 56
18.3.1 Authentication, sessions and role-based access 56
18.3.2 Publication submission 57
18.3.3 Peer and supervisor review 58
18.3.4 Repository search, visibility and analytics 59
18.3.5 Administration, Keycloak sync and reports 59
18.3.6 Email notifications (MailHog) 60
18.4 Summary of Results 60
19. RECOMMENDATIONS FOR FUTURE IMPROVEMENTS 61

Functional and workflow improvements 61
Architecture, scalability and DevOps 61

1. INTRODUCTION

Research article publication is a common practice in the university for PhD and MPhil/Msc students. The students work on theses and dissertations annually and the number increases annually. Despite this increase, there is no system to save all the information for UB lecturers, students and the general public who may want to access these publications outside the conferences and workshops, making it difficult to access these publications. ‘

2. JUSTIFICATION OF PROJECT

Annually, the University of Botswana sees an increasing volume of research articles, thesis and dissertations produced by its academic community. Nevertheless, the lack of a centralized repository hinders UB lecturers, scholars and external stakeholders from easily locating or retrieving these scholarly works. Consequently, implementing a dedicated publication management framework is vital to unify these output assets, providing streamlined search mechanisms, enhanced accessibility and seamless administration. Such a platform will significantly elevate the exposure of UB research while simplifying discoveries for all interested parties.

3. AIM OF THE PROJECT

The main aim of this project is to develop a web-based publication management system that centralizes University of Botswana research outputs and provides users with easy online access to research articles, thesis and dissertations.

4. OBJECTIVES

- Create a system that facilitates the uploading of research articles, theses, and dissertations
- To come up with a system capable of searching of research articles, theses, and dissertations by (author, subject/area, title, abstract, content, etc.),
- To develop a system that allows different users to access information using various query mechanisms.

- To develop a system that allows the review of submitted publications.

5. REQUIREMENTS SPECIFICATION

Requirements Analysis

The established functional requirements follow the project scope. The following phases detail the technical specifications and are split into two parts, the functional requirements and the non-functional requirements to establish the system's quality.

1. Functional Requirements

The functional requirement states what the system shall do by detailing the core functionalities, services, and the tasks it performs.

1. The system shall generate a publications summary report detailing the usage, upload volume, approval status, and publication’s view report.

2. The system shall allow users to manage publications.

3. The system shall allow the system admin to manage user profiles.

4. The system shall allow students, lecturers, and general users to create profiles to be able to log in and retrieve publications.

5. The system shall allow students and lecturers to create(submit) and manage publication records of thesis, dissertations and articles.

6. The system shall allow users (lecturers, students and the general public) to view and/or download publications.

7. The system shall allow lecturers, students and the public to search for and retrieve a publication record.

8. The system shall allow the reviewer to verify and approve publications submitted by lecturers and students.

2. Non-Functional Requirements

The non-functional requirements specification defines the quality attributes and constraints that tell us how the system behaves under those operations.

1. The system must maintain an operational uptime of at least 95% to ensure continuous availability for the University of Botswana lectures, students, and public users.

2. The system should take less than 2 minutes to respond to requests when processing relevant data. It should give out some feedback if it fails to satisfy the user's requests.

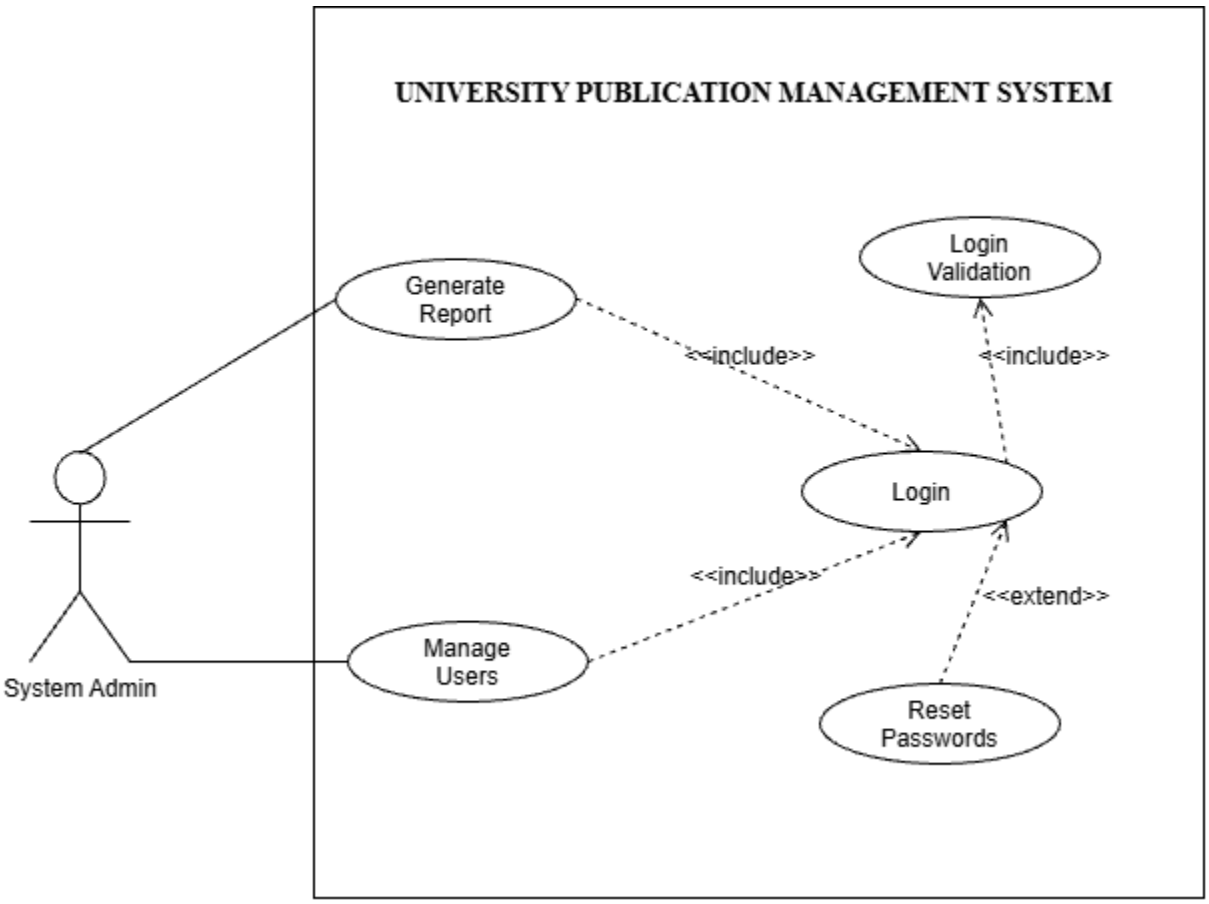
6. USE CASES

Actors identified for our project:

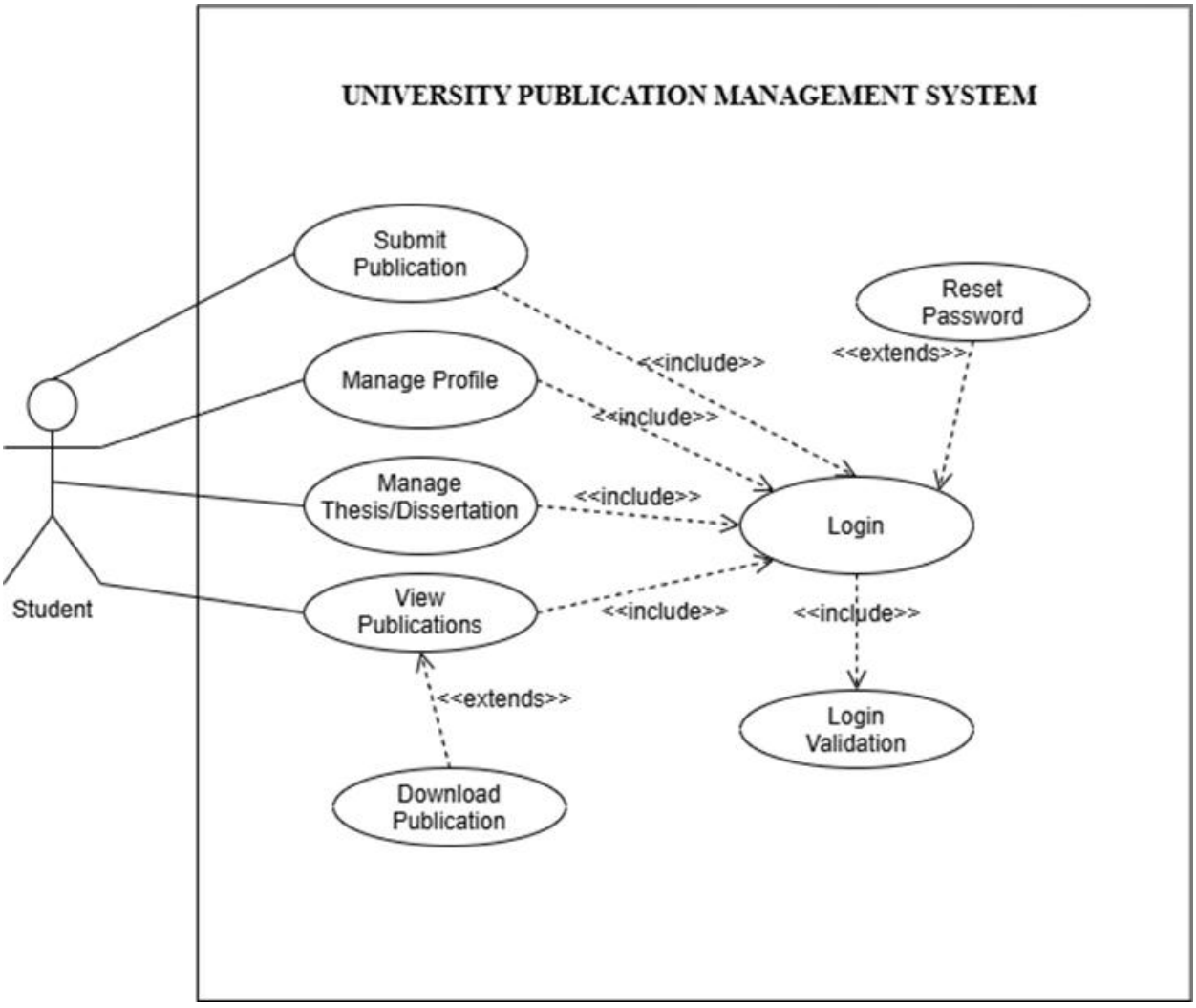
- System Administrator (System Admin)
- Student
- Lecturer
- Reviewer
- General Public

The following are the use case diagrams used:

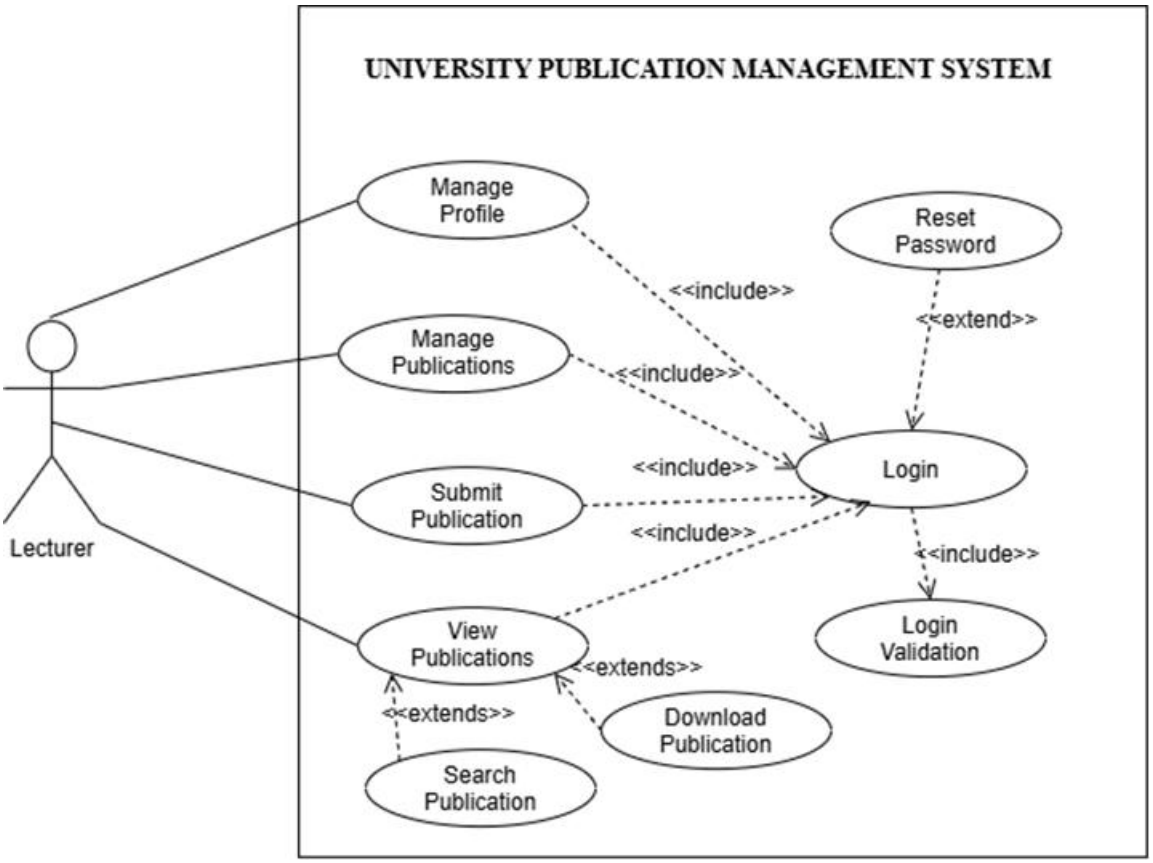
Use Case Diagram-System Admin



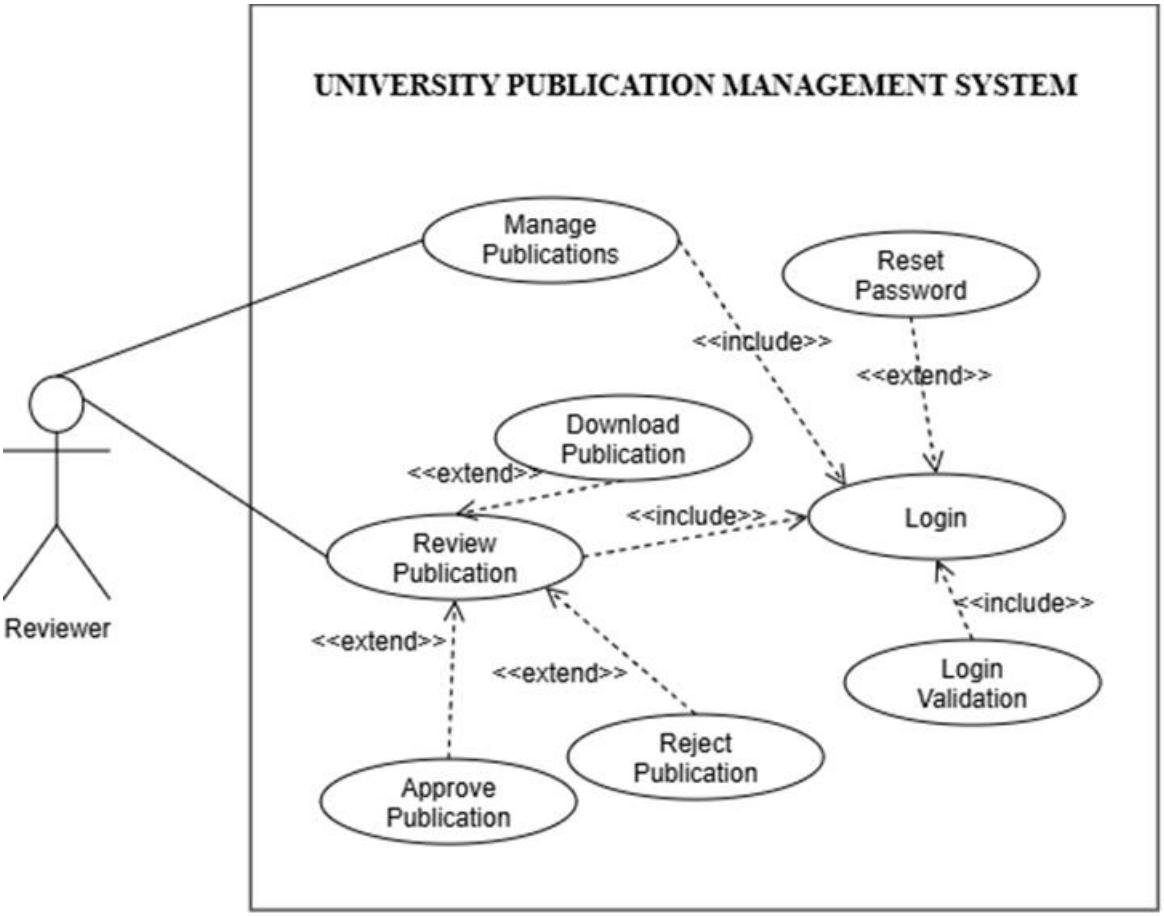
Use Case Diagram-Student



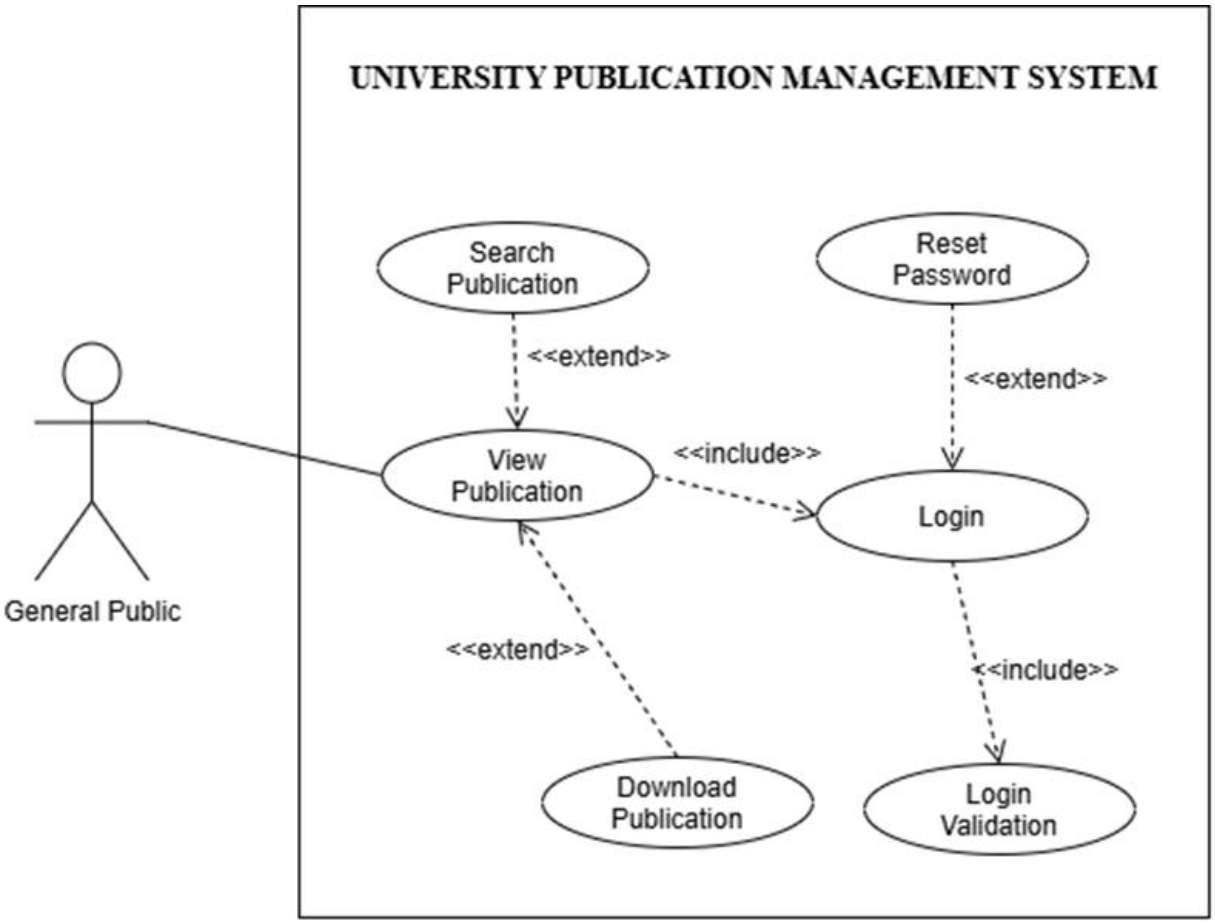
Use Case Diagram-Lecturer



Use Case Diagram-Reviewer



Use Case Diagram-General Public



7. USE CASE SPECIFICATIONS

Specification	Details
NAME OF USE CASE	Login
PARTICIPATING ACTORS	System Admin, Student, Lecturer, Reviewer, General Public
ENTRY CONDITIONS	The use case begins when the user navigates to the login interface.
FLOW OF EVENTS	1. User inputs account credentials (e.g., username/email and password). 2. System invokes **Login Validation** (included). 3. The system grants access and redirects users to their role-specific dashboard.
EXIT CONDITIONS	The use case ends when the user is successfully authenticated and logged into the system.
EXCEPTIONS	Invalid credentials provided; the system presents an error message and permits retry or option to **Reset Password**.
SPECIAL REQUIREMENTS	Encrypted transport (HTTPS) and password hashing.

Specification	Details
NAME OF USE CASE	Login Validation
PARTICIPATING ACTORS	System (Automated)
ENTRY CONDITIONS	The use case begins when a user has submitted login credentials for validation.
FLOW OF EVENTS	1. The system receives submitted credentials from **Login use case**. 2. The System queries the database to verify the existence of the account. 3. The system verifies password hash and checks account active status.
EXIT CONDITIONS	The use case ends when the Credentials verified as valid, returning confirmation to **Login**.
EXCEPTIONS	Database connectivity loss or non-matching credentials which returns failure response.

SPECIAL REQUIREMENTS	Response time under 2 seconds

Specification	**Details**
NAME	Reset Passwords / Reset Password
PARTICIPATING ACTORS	System Admin, Student, Lecturer, Reviewer, General Public
ENTRY CONDITIONS	The use case begins when the User clicks on the "Reset Password" link on the Login interface.
FLOW OF EVENTS	1. User inputs registered email or username. 2. The system sends a secure reset link/token to the user's email. 3. The user accesses the link and submits a new password. 4. System updates credentials and prompts users to log in.

EXIT CONDITIONS	The use case ends when Password updated successfully in the system database.
EXCEPTIONS	Email not found in user directory; system displays an error notice.
SPECIAL REQUIREMENTS	Password recovery links must expire after a set time limit (e.g., 15–30 minutes).

Specification	**Details**
NAME	Generate Report
PARTICIPATING ACTORS	System Admin
ENTRY CONDITIONS	The Use case begins when the System Admin selects the report generation module from the administration panel.

FLOW OF EVENTS	1. System enforces **Login** (included). 2. Admin selects parameters (e.g., publication stats, submission volumes, date ranges). 3. System queries system data and compiles the report structure. 4. The system displays summaries and offers export options.
EXIT CONDITIONS	The use case ends when the System report is generated and available for viewing/downloading.
EXCEPTIONS	Database timeout or zero records match requested parameter set.
SPECIAL REQUIREMENTS	Capability to export generated reports to PDF or CSV format.

Specification	**Details**
NAME	Manage Publications

PARTICIPATING ACTORS	System Admin, Lecturer
ENTRY CONDITIONS	The use case begins when the Admin or Lecturer accesses the administrative publication control section.
FLOW OF EVENTS	1. System enforces **Login** (included). 2. User views overall publication records. 3. User creates, edits metadata, updates status, or removes publication entries as authorized.
EXIT CONDITIONS	The use case ends when changes to publication records are stored in the system database.
EXCEPTIONS	Requested record locked by another process or deletion fails due to dependency.
SPECIAL REQUIREMENTS	Maintain modification log for auditing changes.

Specification	Details
NAME	Manage Users
PARTICIPATING ACTORS	System Admin
ENTRY CONDITIONS	The use case begins when the System Admin selects the user management section from the menu.
FLOW OF EVENTS	1. System enforces **Login** (included). 2. Admin views list of system users. 3. Admin creates new profiles, edits user attributes/roles, or deactivates user accounts.
EXIT CONDITIONS	The use case ends when a User account modifications saved successfully in the system repository.
EXCEPTIONS	Primary admin account cannot be deleted or deactivated.
SPECIAL REQUIREMENTS	Role-based permission assignments (Admin, Lecturer, Student, Reviewer).

Specification	Details
NAME	Submit Publication
PARTICIPATING ACTORS	Student, Lecturer
ENTRY CONDITIONS	The use case begins when the Student or Lecturer selects "Submit Publication" on the dashboard.
FLOW OF EVENTS	1. System enforces **Login** (included). 2. User enters manuscript metadata (title, abstract, authors, domain keywords). 3. The user uploads a publication document file. 4. The user submits items for processing/reviews
EXIT CONDITIONS	The use case ends when the Publication submitted with initial status logged in the system.
EXCEPTIONS	File type not supported or file exceeds maximum allowed size.

SPECIAL REQUIREMENTS	File upload size limits are strictly enforced (e.g., maximum 100MB PDF).

Specification	**Details**
NAME	Manage Profile
PARTICIPATING ACTORS	Student, Lecturer
ENTRY CONDITIONS	The use case begins when Student or Lecturer accesses personal profile settings.
FLOW OF EVENTS	1. System enforces **Login** (included). 2. User views current profile details (contact info, department, affiliation). 3. User updates fields and submits updates.
EXIT CONDITIONS	The use case ends when the Profile details are successfully updated in the system.

EXCEPTIONS	System rejects invalid input formats (e.g., malformed email or telephone).
SPECIAL REQUIREMENTS	Read-only restrictions on university registration/employee IDs.

Specification	**Details**
NAME	Manage Thesis / Dissertation
PARTICIPATING ACTORS	Student
ENTRY CONDITIONS	The use case begins when the Student enters the academic thesis submission section.

FLOW OF EVENTS	1. System enforces **Login** (included). 2. The student selects the thesis/dissertation category. 3. Student uploads draft/final document or updates submission details.
EXIT CONDITIONS	The use case ends when Thesis/dissertation document is saved under student record.
EXCEPTIONS	Submission deadline elapsed; system locks modification form.
SPECIAL REQUIREMENTS	Mandatory PDF document upload standard.

Specification	**Details**
NAME	Search Publication

PARTICIPATING ACTORS	Lecturer, General Public
ENTRY CONDITIONS	The use case starts when the lecturer is viewing publications and searches for a publication.
FLOW OF EVENTS	1. The user enters keywords, author name, title, or publication category into the search bar. 2. System filters repository based on criteria. 3. System renders matching results list to screen.
EXIT CONDITIONS	The use case ends when Filtered search results are rendered to the user.
EXCEPTIONS	No matching publication records returned.
SPECIAL REQUIREMENTS	Results are displayed in under 10 seconds.

Specification	Details
NAME	Review Publication
PARTICIPATING ACTORS	Reviewer
ENTRY CONDITIONS	The use case begins when the Reviewer views the assigned publications queue.
FLOW OF EVENTS	1. System enforces **Login** (included). 2. Reviewer selects an assigned publication to evaluate. 3. Reviewers may trigger **Download Publication** (extension) to inspect file. 4. Reviewer provides evaluation comments. 5. Reviewer invokes **Approve Publication** or **Reject Publication** (extensions)
EXIT CONDITIONS	The use case ends when Review evaluation and decision recorded in system.
EXCEPTIONS	Assigned publication reassigned or recalled prior to review completion.
SPECIAL REQUIREMENTS	

Specification	Details
NAME	Approve Publication
PARTICIPATING ACTORS	Reviewer
ENTRY CONDITIONS	The use case begins when a reviewer is reviewing a publication and selects approve.
FLOW OF EVENTS	1. The reviewer selects the "Approve" option during the review process. 2. System updates publication status to "Approved" or "Published". 3. The system triggers notification to the author.
EXIT CONDITIONS	The use case ends when Publication is marked as approved in the database.

EXCEPTIONS	Required rating or feedback fields left incomplete by reviewer.
SPECIAL REQUIREMENTS	Automated notification dispatch to submitter upon approval.

Specification	**Details**
NAME	Reject Publication
PARTICIPATING ACTORS	Reviewer
ENTRY CONDITIONS	The use case begins when a reviewer is reviewing a publication and selects reject publication.
FLOW OF EVENTS	1. The reviewer selects the "Reject" option during the review process. 2. Reviewer enters required rationale or revision requirements. 3. System updates publication status to "Rejected" or "Revision Required".

EXIT CONDITIONS	The use case ends when Rejection status and feedback is saved in the database.
EXCEPTIONS	Rejection justification text field left blank.
SPECIAL REQUIREMENTS	Mandatory feedback text field required prior to final rejection submission.

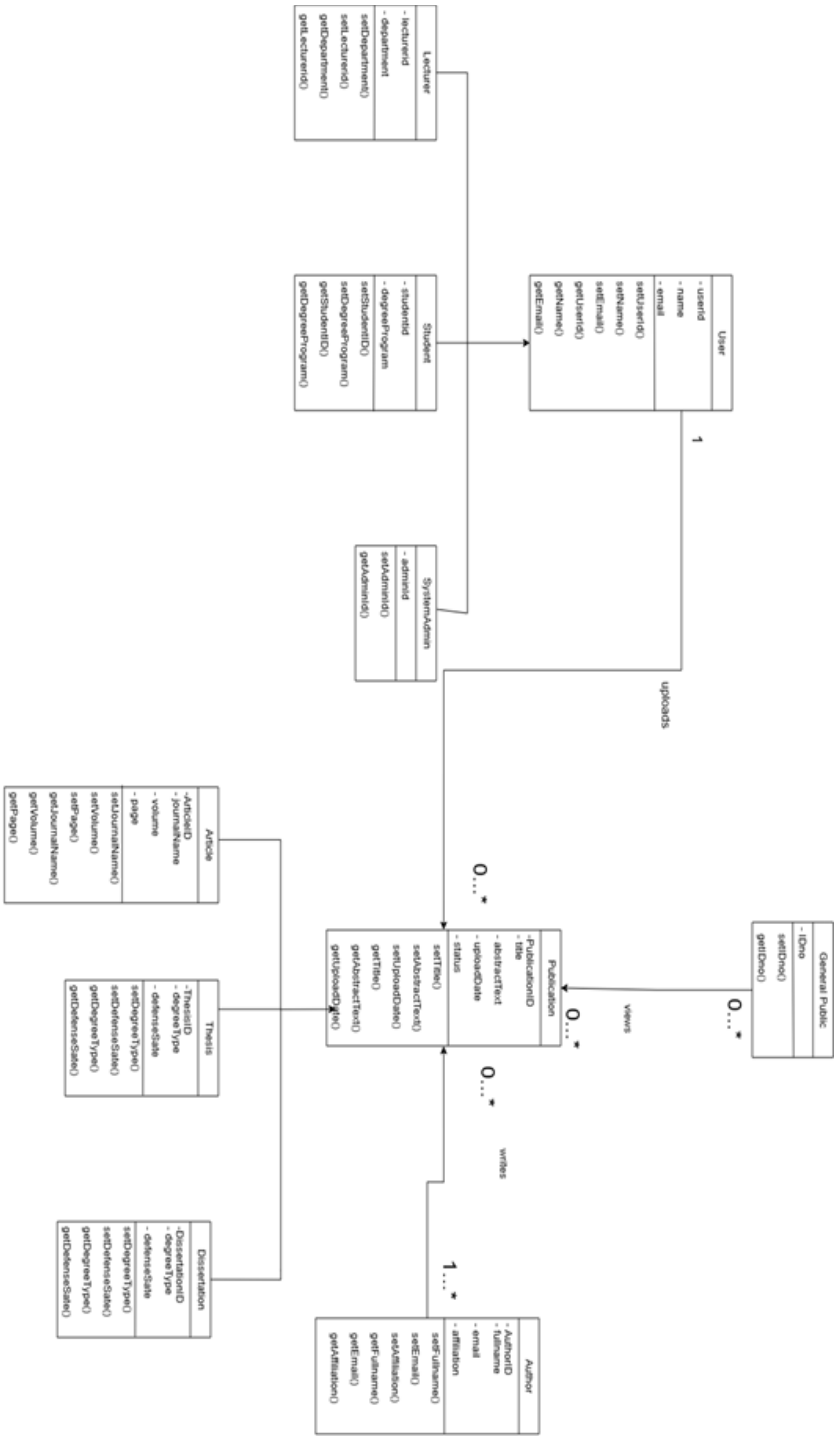
Specification	**Details**
NAME	View Publications / View Publication
PARTICIPATING ACTORS	Student, Lecturer, General Public
ENTRY CONDITIONS	The use case begins when a user opens a repository page or selects a public catalog.

FLOW OF EVENTS	1. System enforces **Login** (included). 2. The system displays a catalog list of published papers. 3. The user selects a paper to view 4. Users may optionally invoke **Search Publication** (extension) or **Download Publication** (extension).
EXIT CONDITIONS	The use case ends when a selected publications details are displayed on UI.
EXCEPTIONS	Publication set to confidential or restricted view.
SPECIAL REQUIREMENTS	Responsive layout across desktop and mobile browsers.

Specification	**Details**
NAME	Download Publication
PARTICIPATING ACTORS	Student, Lecturer, Reviewer, General Public

ENTRY CONDITIONS	The use case begins when a user clicks downloads publication on the view publications page
FLOW OF EVENTS	1. The user clicks the "Download" button on the publication detail page. 2. System checks download permissions/access rights for files. 3. System streams file payload to client browser/device.
EXIT CONDITIONS	The use case ends when the Publication file is downloaded to user local storage.
EXCEPTIONS	File missing on storage server or network connectivity interrupted.
SPECIAL REQUIREMENTS	The publications should be downloaded as pdf's

8. INITIAL DOMAIN MODEL



9. REQUIREMENTS PRIORITIZATION

High Priority:

1. The system shall allow students and lecturers to create(submit) and manage publication records of thesis, dissertations and articles. [High]
2. The system shall allow lecturers, students and the public to search for and retrieve a publication record. [High]
3. The system shall allow the reviewer to verify and approve publications submitted by lecturers and students.[High]

Medium Priority:

1. The system shall allow the system admin to manage user profiles.[Medium]
2. The system shall allow students, lecturers, and general users to create profiles to be able to log in and retrieve publications.[Medium]
3. The system shall allow users to manage publications.[Medium]

Low Priority:

1. The system shall allow users (lecturers, students and the general public) to view and/or download publications. [Low]
2. The system shall generate a publications summary report detailing the usage, upload volume, approval status, and publication's view report.[Low]

10. DESIGN SPECIFICATION

10.1 Design Goals

Client / Customer (University of Botswana Management)

- **Cost-effectiveness:** Minimize deployment and operational expenses by leveraging existing University of Botswana server infrastructure rather than purchasing third-party services.
- **Rapid development:** Build and deploy the functional system within the strict academic project timeline.

End User (Lecturers, Students, Reviewers, System Admin, & General Public)

- **Reliability:** Maintain an operational uptime of at least 95% to ensure continuous availability for university members and public users (NFR 1).
- **High performance:** Execute searches, process relevant data, and return query responses in under 2 minutes (NFR 2).

Developer / Maintainer

- **Maintainability:** Organize business logic within a 3-layer structure so future changes to university approval policies or database schemas require minimal effort.
- **Modifiability:** Decouple system partitions so modifying one module (e.g., Report Generation) does not ripple through or break others (e.g., User Management).
- **Well-defined interfaces:** Publish explicit, typed service interfaces for each subsystem (User Management, Review, Submission, Repository Access) to achieve low coupling.

10.2 Subsystem Responsibilities

User Interface Subsystem: The subsystem is positioned in the presentation layer and is appropriately named, as it focuses on presenting system functionality and receiving user input. Therefore, it has a single responsibility, which shows how different clients see and use the system. This subsystem is also modular and decoupled from other subsystems in that its swapping will not affect other subsystems.

User Authentication Subsystem: The subsystem manages the users' access to the system and also manages the user's session data. It is modular and decoupled from other systems, and it won't affect neighbouring subsystems.

Publication Management Subsystem: The subsystem has the sole responsibility of creating, searching, retrieving, publishing, and generating reports based on publication management. This subsystem is decoupled from other subsystems in the system.

User Management Subsystem: The subsystem's name is appropriate and articulates the responsibility of managing the user's profile information. It is highly decoupled from other subsystems in that its swapping will not affect the functionality of other subsystems in the system.

Database Management Subsystem: The subsystem has the single responsibility of securing, storing, and retrieval of data stored in the repository. It is highly decoupled and highly modular, as it will not affect any other subsystem's functionality.

10.3 Architectural Style Overview

The system employs a **Hybrid Architectural Pattern** combining **Layered**, **Client-Server**, and **Repository** architectural styles to achieve clear separation of concerns, scalability, and centralized data governance:

- Client-Server Style: Web clients (Students, Lecturers, General Public) reside on client devices, while business processing and data access are centralized on the University of Botswana server infrastructure.

- Layered Style: Functionality is strictly divided into three sequential layers: **Presentation Layer** (Web UI & Auth), **Application / Business Logic Layer** (Publication, Report, and User Management), and **Database / Data Link Layer** (Database Management).

- Repository Style: Dictates the **data storage and sharing strategy**. A central database serves as the single source of truth for all research publications, user records, and

system logs. All independent business subsystems communicate through this central data store via the **Database Management Subsystem**.

·al Style	·eflected in the Diagram	Design Goals
	nto Presentation, /Business Logic, and ata Link layers.	**ility & Modifiability:** Isolates business rules so changes in licies or UI designs don't force schema changes in the database. **:** Clear layer boundaries allow developers to build, review, and test in parallel.
er	n client apps (Student, iblic) connected over the central backend server.	: The separation of the client and the serve makes it easier to control ect data and protect the system **liness, Ease of Use, and Ease of Learning** because the client can mple interface while the server handles the complex processing in und.
	database and Database t Subsystem serving all modules.	**& Consistency:** Provides a single, authoritative store for and user data, ensuring 95%+ uptime and continuous access. **:** Centralizes file-streaming, database access routines, and data ules for all subsystems.

Why the Architecture and Subsystems Support our Design Goals

Presentation Layer Subsystems

- **User Interface Subsystem:**
    a. Alignment*:* **User-friendliness, Ease of Use, Ease of Learning.**

    b. Why it fits: Delivers role-tailored views for Students, Lecturers, and the General Public. Designed so guest readers can search and view publications immediately without formal training, and users can submit or approve documents quickly (under 10 minutes).
- **User Authentication Subsystem (Keycloak IAM):**

a. Alignment: **Reusability & Well-Defined Interfaces.**

b. Why it fits: Delegates Identity and Access Management to Keycloak IAM. Standardizes user verification across all client roles without cluttering core business logic.

Application / Business Logic Layer Subsystems

- **Publication Management Subsystem:**
  a. Alignment: **Robustness & Rapid Development.**

  b. Why it fits: Encapsulates submission and review workflows. Handles file checks, validates metadata, and catches invalid or none processable uploads before they reach the database, delivering explicit error messages to the user.
- **Report Generation Subsystem:**
  a. Alignment: **Modifiability & High Performance.**

  b. Why it fits: Isolated from the Publication Subsystem so complex analytics and query aggregation do not degrade the performance of core publication uploads or search responses (keeping operations well under 2 minutes)

- **User Management Subsystem:**
  a. Alignment: **Maintainability & Modifiability.**
  b. Why it fits: Exposes explicit service interfaces for profile management and permissions. Decoupled so changes to user roles or university hierarchy do not break publication or reporting tools.

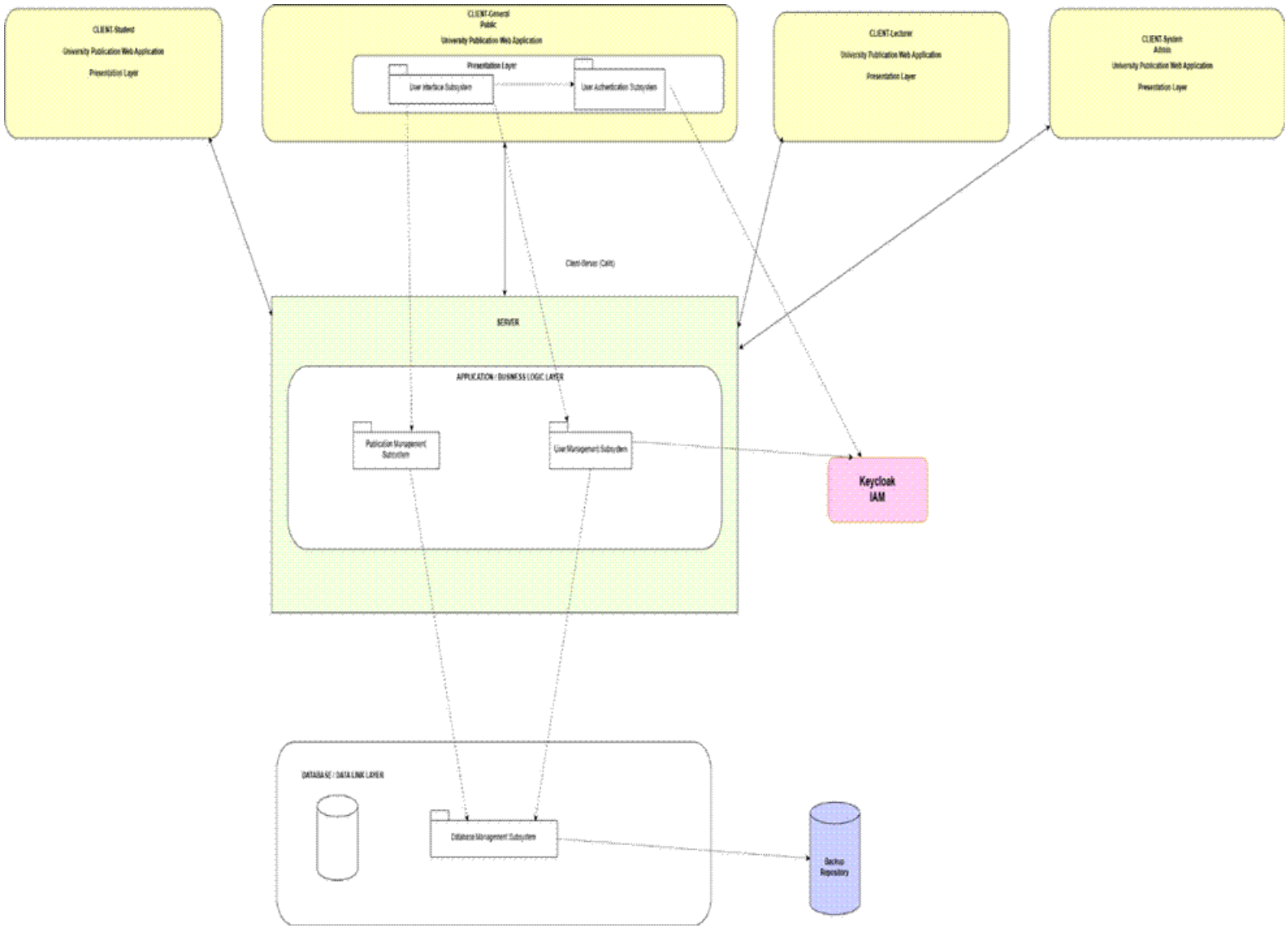
Database / Data Link Layer Subsystem

- **Database Management Subsystem:**
  a. Alignment: **Reliability, Cost-Effectiveness, & Reusability.**

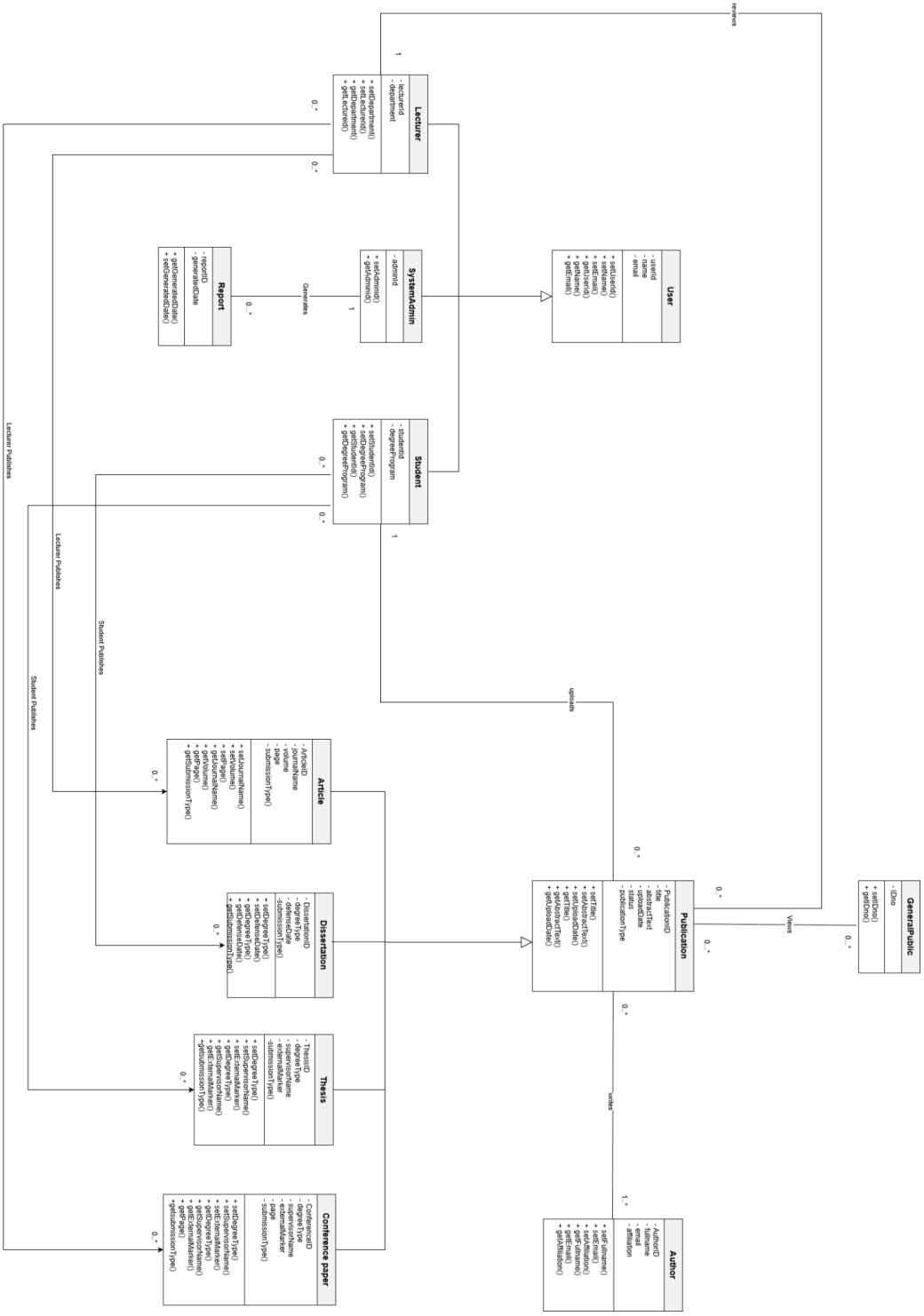
  b. Why it fits: Manages central repository access, indexing, and connection pooling. Runs directly on University of Botswana server infrastructure to

eliminate hosting costs while delivering continuous availability and 95%+ operational uptime

11. ARCHITECTURE

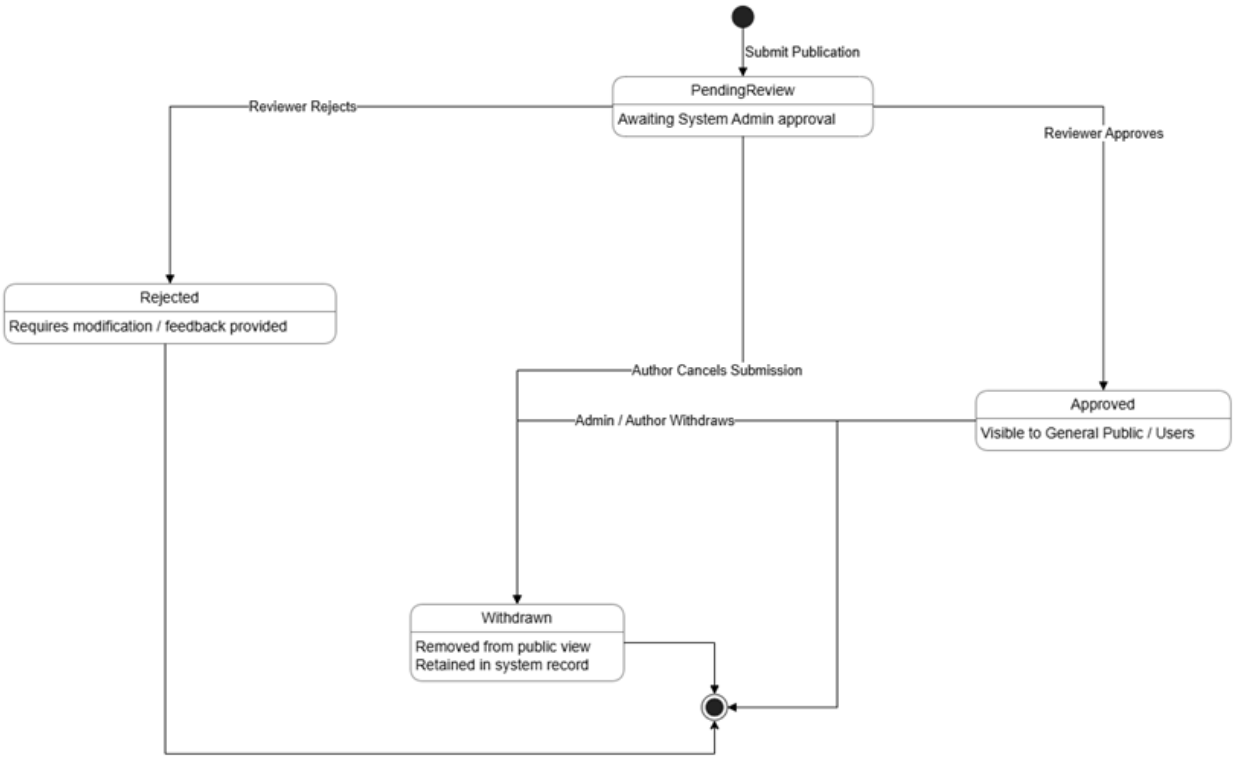

CLIENT-Student
University Publication Web Application
Presentation Layer
CLIENT-General Public
University Publication Web Application
Presentation Layer
User Interface Subsystem
User Authentication Subsystem
CLIENT-Lecturer
University Publication Web Application
Presentation Layer
CLIENT-System Admin
University Publication Web Application
Presentation Layer
Client-Server (Café)
SERVER
APPLICATION / BUSINESS LOGIC LAYER
Publication Management Subsystem
User Management Subsystem
Keycloak IAM
DATABASE / DATA LINK LAYER
Database Management Subsystem
Backup Repository

12. Object Domain Model



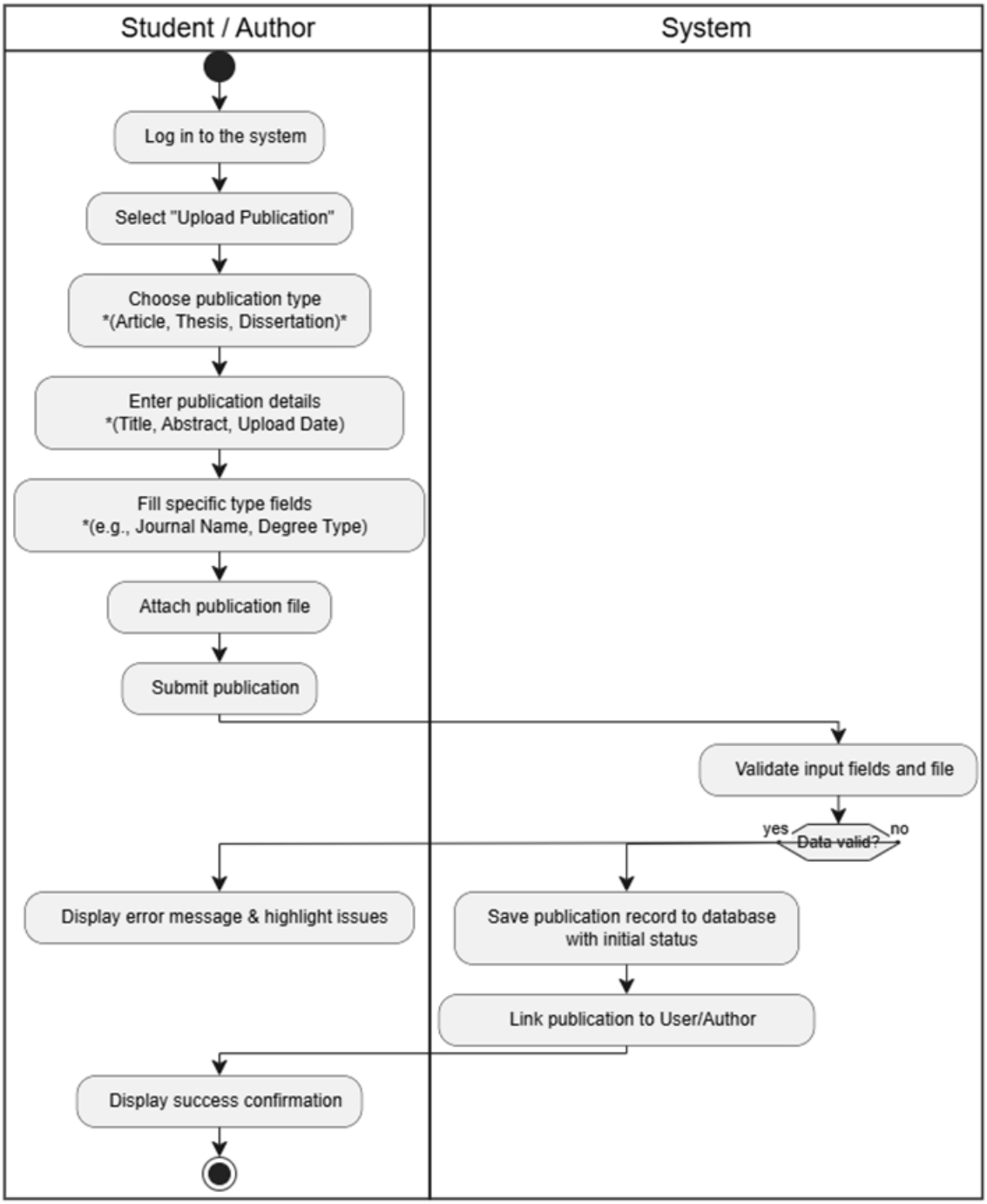
13. State Chart Diagram

13.1 Publication State Chart

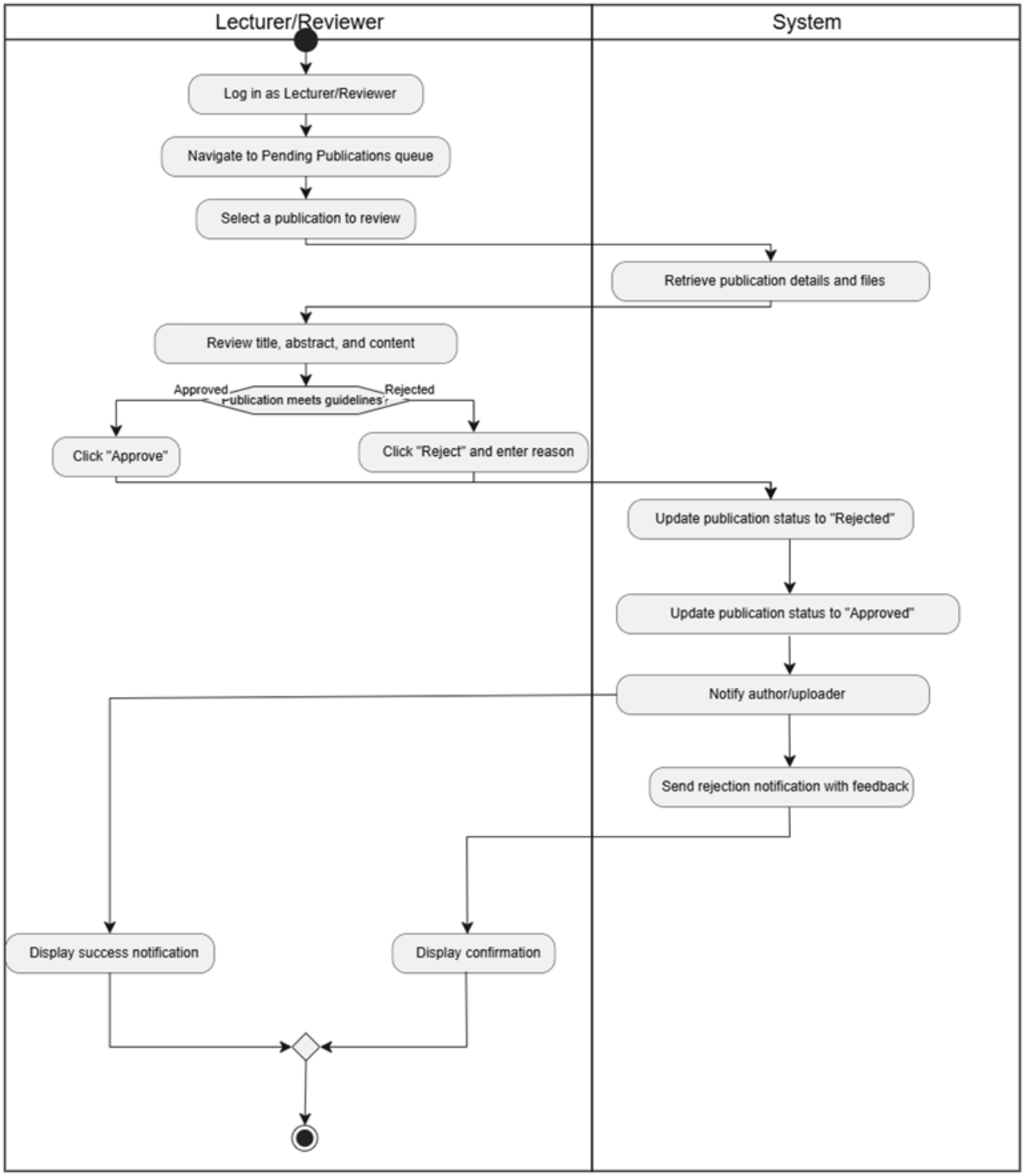


14. Activity Diagrams

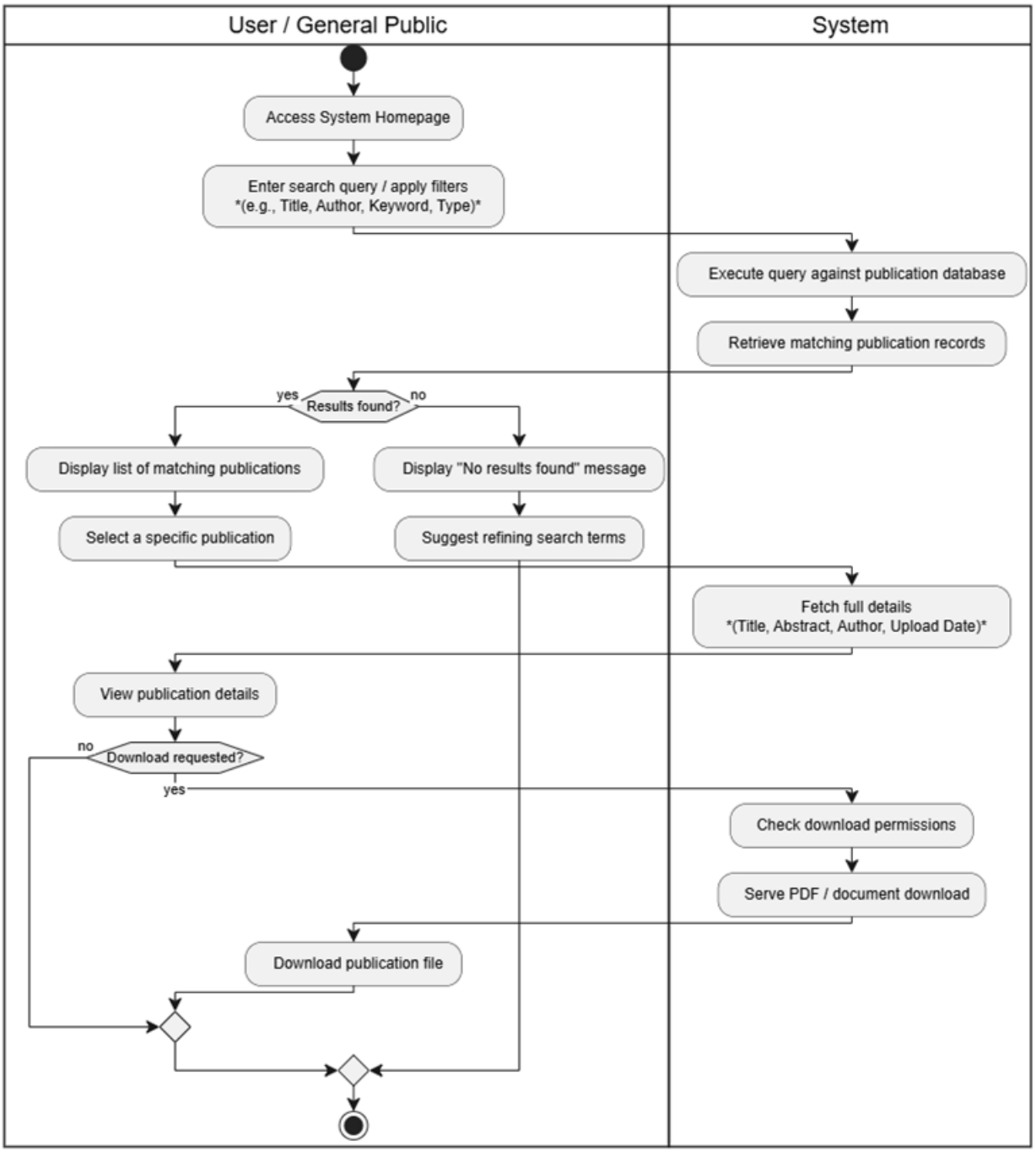
14.1 Upload Publication Activity Diagram



14.2 Review Publication Activity Diagram



14.3 Search/Download Activity Diagram



15. IMPLEMENTATION AND FILE STRUCTURE

The project follows a client-server structure. The Angular frontend provides the user interface, while the Node.js backend processes requests, applies business rules, communicates with the database and supports file uploads and email notifications. Keycloak provides authentication and role-based access control.

15.1 Simplified File Structure

```
PROJECT ROOT/
|
|-- api/                        # Additional Node.js API service
|   |-- server.js               # Starts the API service
|   `-- package.json                    # API dependencies and scripts
|
|-- backend/                    # Main Node.js backend
|   |-- src/
|   |   |-- config/
|   |   |   |-- database.js     # Database connection configuration
|   |   |   `-- keycloak.js     # Keycloak integration configuration
|   |   |-- controllers/
|   |   |   `-- publication.controller.js  # Publication request handling
|   |   |-- middleware/
|   |   |   |-- auth.middleware.js   # Verifies authentication tokens
|   |   |   |-- role.middleware.js   # Enforces user roles
|   |   |   `-- upload.middleware.js # Handles uploaded files
|   |   |-- routes/
|   |   |   |-- auth.routes.js     # Authentication endpoints
|   |   |   `-- publication.routes.js # Publication endpoints
|   |   |-- services/
|   |   |   `-- email.service.js            # Sends email notifications
|   |   `-- server.js             # Starts the backend server
|   |-- .env                    # Backend environment variables
|   `-- package.json                    # Backend dependencies and scripts
|
|-- keycloak/
|   `-- realm-export.json               # Realm, clients, roles and users configuration
|
|-- src/                        # Angular frontend
|   |-- app/
|   |   |-- models/
|   |   |   `-- publication.model.ts # Publication data structure
|   |   |-- Publication Management Subsystem/
|   |   |   |-- review-publication/  # Lists publications for review
|   |   |   |-- review-publication-detail/ # Shows review details
|   |   |   |-- Submit Publication/  # Publication submission feature
|   |   |   `-- view-search-publication/ # Search and view feature
```

```
|  |  |-- services/
|  |  |  |-- auth.service.ts    # Frontend authentication operations
|  |  |  |-- auth.interceptor.ts  # Adds tokens to HTTP requests
|  |  |  |-- publication.service.ts # Calls publication API
|  |  |  `-- role.guard.ts      # Protects routes by role
|  |  |-- User Interface Subsystem/
|  |  |  |-- Dashboard/        # General and student dashboards
|  |  |  `-- Login/             # Login page and logic
|  |  |-- not-found/            # Invalid-route page
|  |  |-- app.component.ts              # Root component logic
|  |  |-- app.component.html            # Root component template
|  |  |-- app.config.ts         # Application-wide providers
|  |  `-- app.routes.ts         # Main frontend routes
|  |-- assets/images/                   # Static images and logo
|  |-- index.html               # Page hosting the Angular application
|  |-- main.ts                  # Angular entry point
|  `-- styles.css               # Global styles
|
|-- angular.json                # Angular build configuration
|-- package.json                        # Main project dependencies and scripts
|-- tsconfig.json               # TypeScript compiler configuration
|-- docker-compose.yml                  # Container service configuration
|-- database.sql / db.sql               # Database scripts
`-- Publications_API.postman_collection.json # API test collection
```

15.2 Angular Frontend

The src folder contains the Angular user interface. It controls what users see and how they interact with the system.

- Components combine TypeScript logic, HTML templates and CSS or SCSS styling.
- The publication management components support submission, searching, viewing and reviewing publications.
- Angular services communicate with the backend API and manage shared functions such as authentication.
- app.routes.ts maps browser paths to components, while role.guard.ts prevents unauthorised navigation.

15.3 Node.js API and Backend

The api and backend folders contain server-side JavaScript. The backend receives HTTP requests from Angular, processes them and returns responses.

- server.js creates and starts the Node.js server and registers middleware and routes.
- package.json records backend libraries and commands for starting or developing the server.
- .env stores environment-specific values such as ports, database settings and authentication configuration. It should not be committed with real secrets.

15.4 Configuration Files

The backend/src/config folder centralises external system settings.

- database.js establishes and manages the connection to the project database.
- keycloak.js configures communication between the backend and Keycloak.

15.5 Controllers

Controllers contain the main request-handling logic. publication.controller.js receives publication requests from the routes, validates or processes the supplied data, calls the required database operations and sends an HTTP response to the frontend.

15.6 Middleware

Middleware runs between the incoming request and the final controller.

- auth.middleware.js verifies that the request contains a valid authentication token.
- role.middleware.js checks whether the authenticated user has the required role or permission.
- upload.middleware.js processes upload publication documents before the controller handles them.

15.7 Routes

Routes define the backend endpoints that the Angular frontend can call.

- auth.routes.js provides authentication-related endpoints.
- publication.routes.js provides endpoints for submitting, retrieving, searching, reviewing or updating publications.
- Routes forward valid requests to middleware and then to the appropriate controller.

15.8 Keycloak Authentication

The keycloak folder contains realm-export.json. This file stores the exported Keycloak realm configuration, including clients, roles and authentication settings. Angular obtains and sends access tokens, the backend validates them, and role checks determine which functions a user may access.

15.9 Notification Service

The backend/src/services/email.service.js file provides the notification functionality identified in the repository. It is used to send email notifications about relevant publication events, for example submission or review status changes. The controller can call this service after the relevant backend action succeeds.

15.10 Database and Supporting Files

Database scripts define or reproduce the project database, while supporting configuration files help run and test the system.

- database.sql, db.sql and iteration2dbdump.sql contain database definitions or saved database data.
- docker-compose.yml defines the services that can run together in containers, such as the application, database and Keycloak.
- Publications_API.postman_collection.json contains saved requests for testing the API in Postman.
- angular.json and tsconfig.json control Angular building and TypeScript compilation.

15.11 Files Excluded from the Documentation

The expanded node modules directories, generated Angular cache files, uploaded documents, lock-file internals and individual test files are excluded from the simplified structure. They are not primary source-code modules and would make the documentation unnecessarily long.

15.12 Request Flow

A typical request flows through the system as follows:

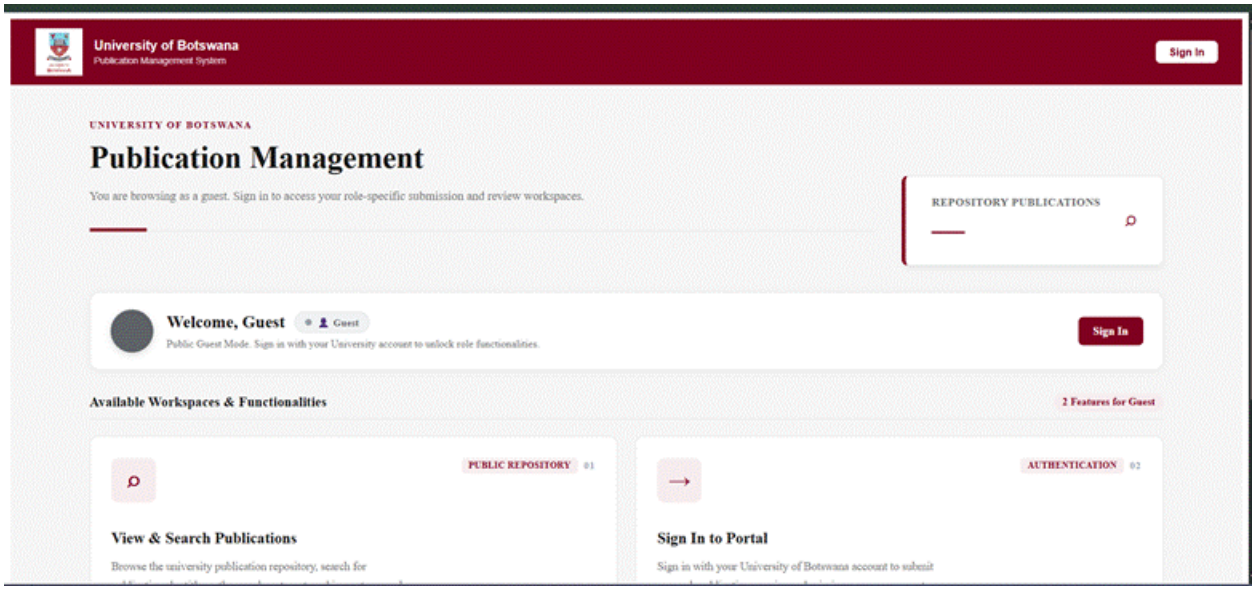
1. The user performs an action in an Angular component.
2. An Angular service sends an HTTP request to a backend route.
3. Authentication and role middleware validate the request.
4. The route forwards the request to the publication controller.
5. The controller performs database or file-processing operations.
6. The email service sends a notification when required.
7. The backend returns a response and Angular updates the interface.

16. TEAM RESPONSIBILITIES

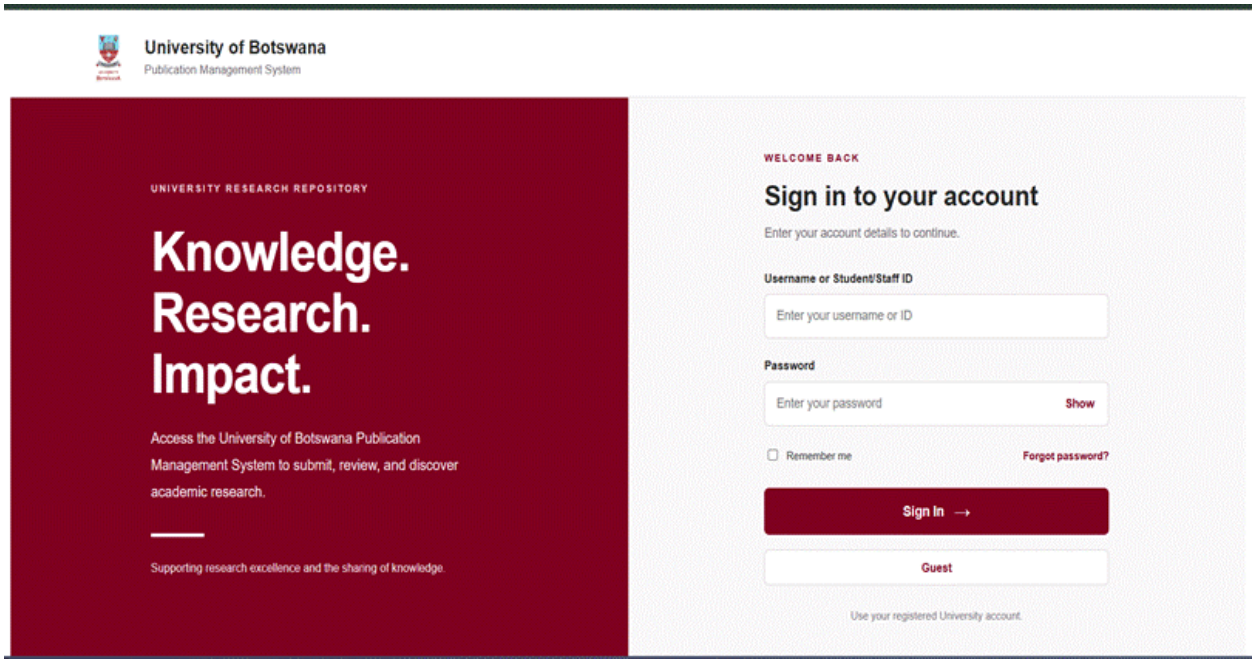
- **Keletso Joy Ditlhotlhole – Group Leader / Frontend Development**
  Served as the group leader, coordinating team activities and ensuring an organised development process, including maintaining order within the shared repository. She contributed to UI design and frontend implementation and was specifically responsible for preparing the documentation and repository structure summary for the project documentation.
- **Kgosi Zilwa – Authentication and Access Control**
  Primarily responsible for implementing authentication and role-based access control, ensuring that users had the appropriate permissions to access system pages and core functionality, particularly administrator features and restricted pages.
- **Phazha Chakalisa – Backend Development**
  Primarily contributed to the development and implementation of backend functionality supporting the system's core features.
- **Archibald Mareme – Backend Development**
  Primarily contributed to backend development and integration**,** ensuring that server-side functionality supported the required system operations.
- **Kagiso Mpopang – Backend Development**
  Primarily contributed to the implementation and integration of backend functionality required by the system.
- **Entire Team – Documentation, Backend, Integration and Maintenance**
  The entire team collaboratively contributed to project documentation, backend development, system integration and maintenance. Team members participated in testing, troubleshooting, resolving implementation issues, refining functionality, and ensuring that the frontend, backend, authentication, database and other system components worked together correctly.

17. OVERVIEW OF UI

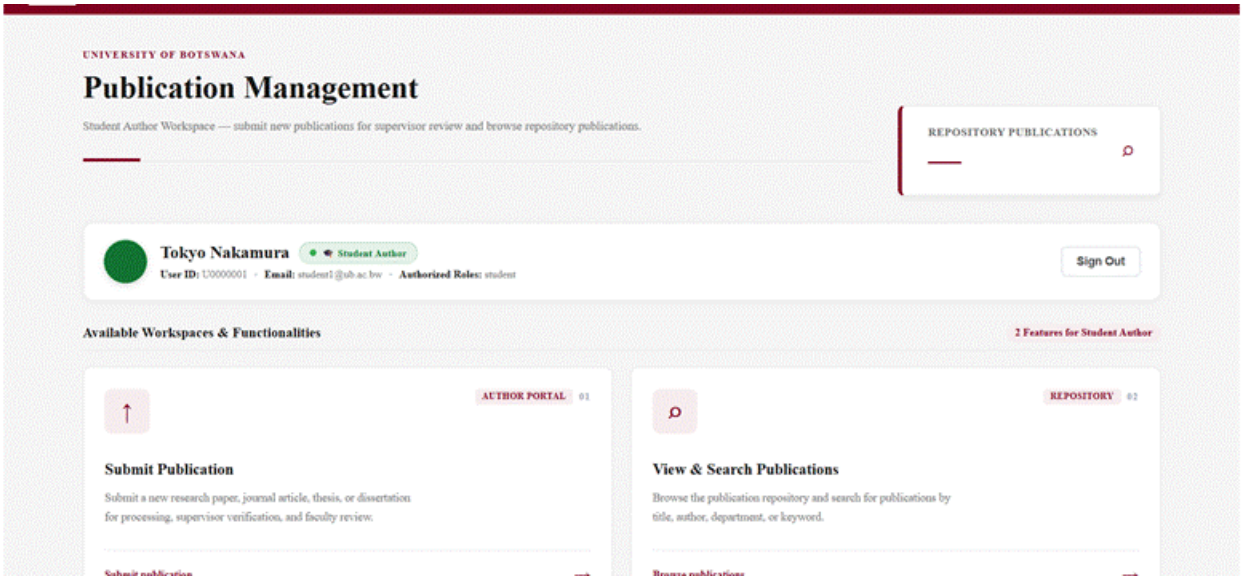
Guest Dashboard



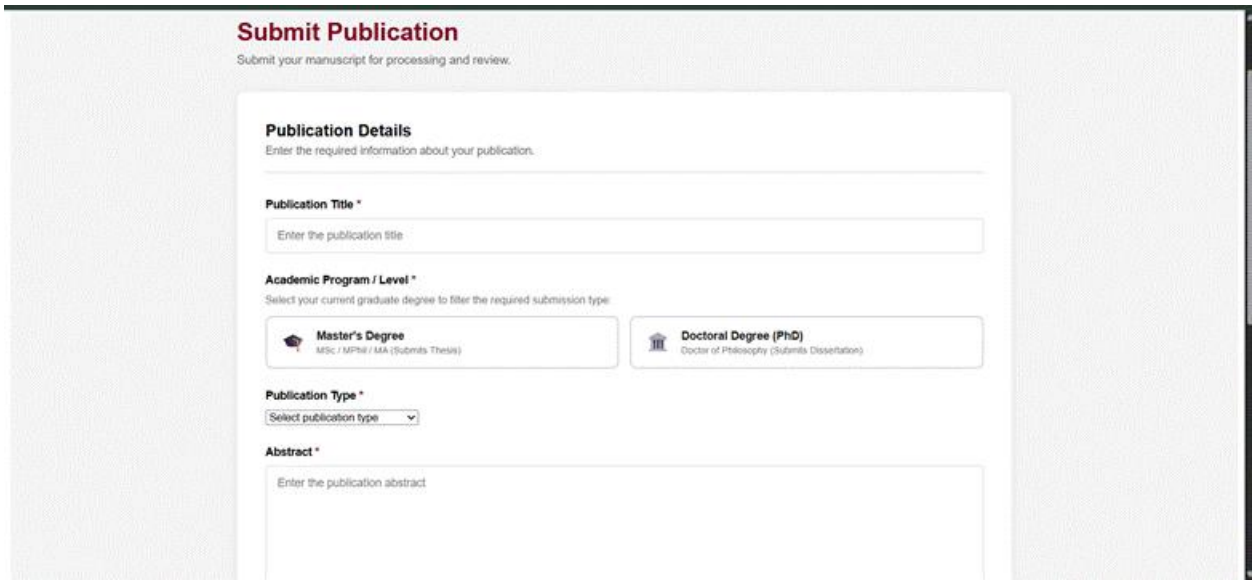
Login page

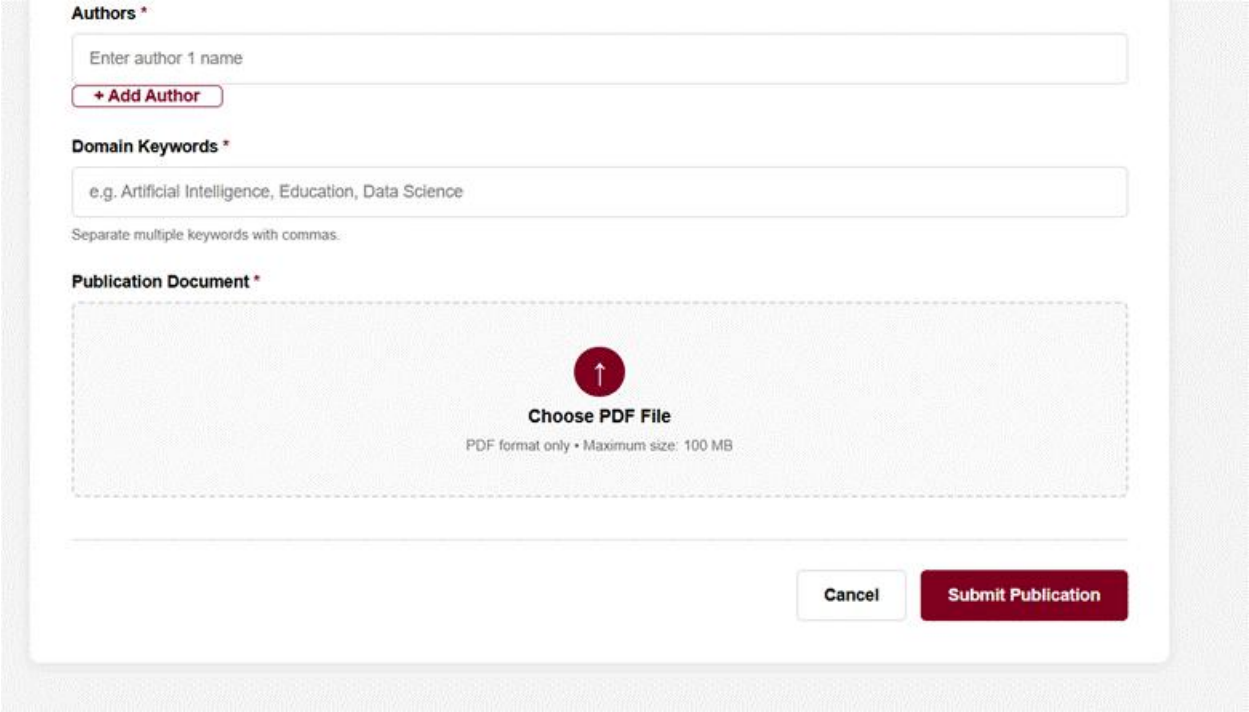


Student dashboard

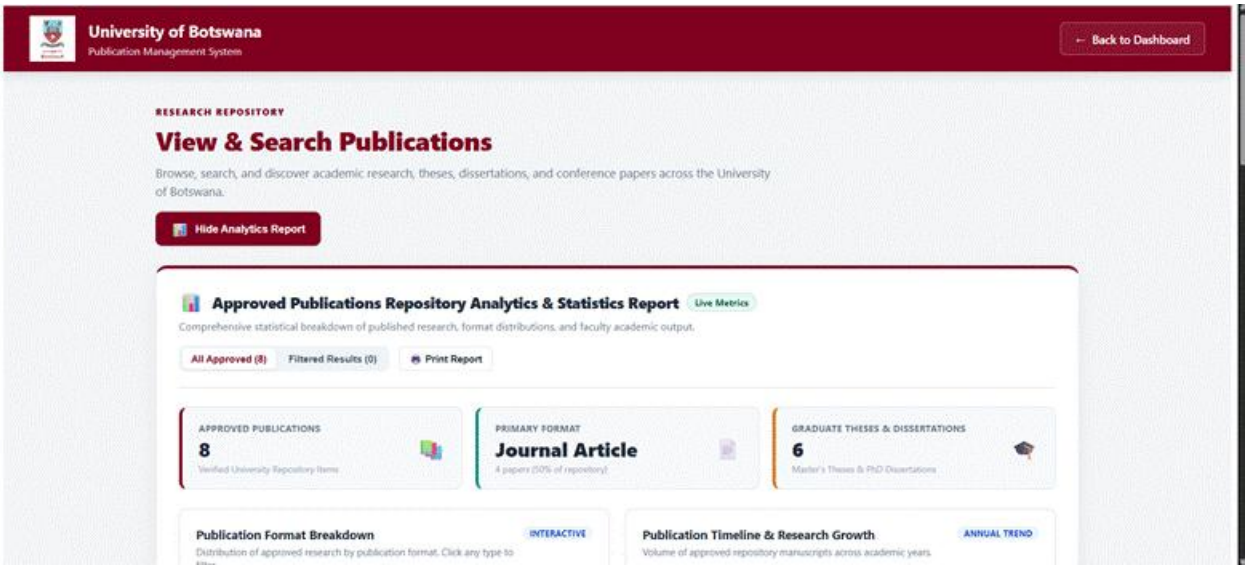


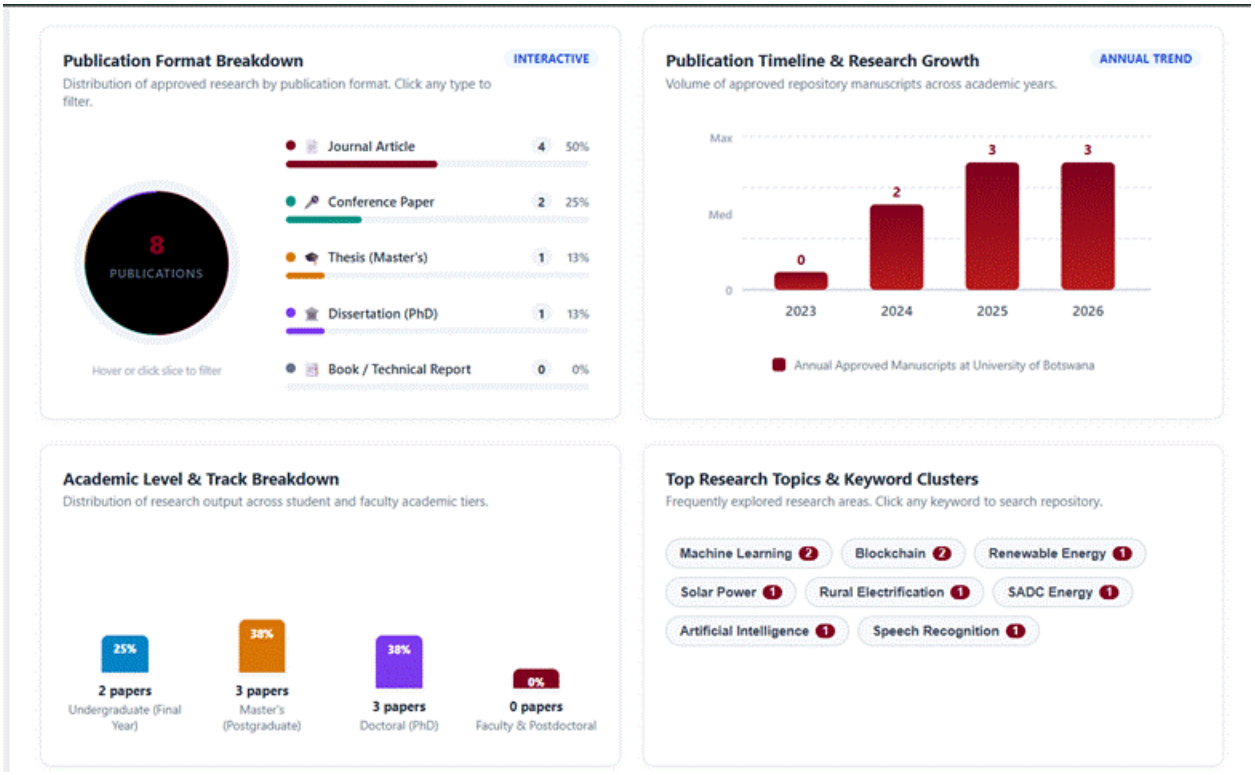
Student submit publication page





View and Search publication page(student)

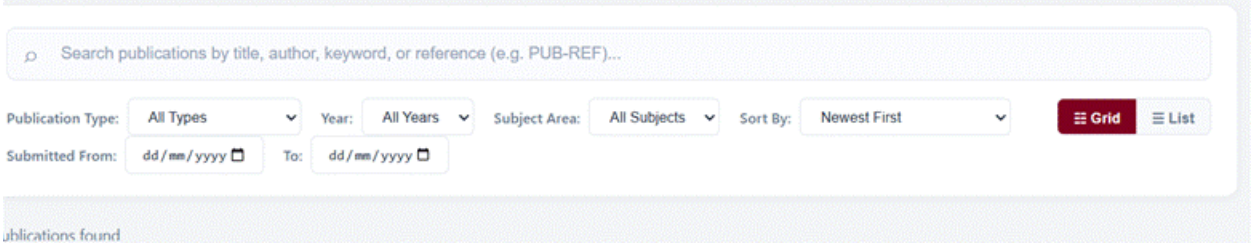




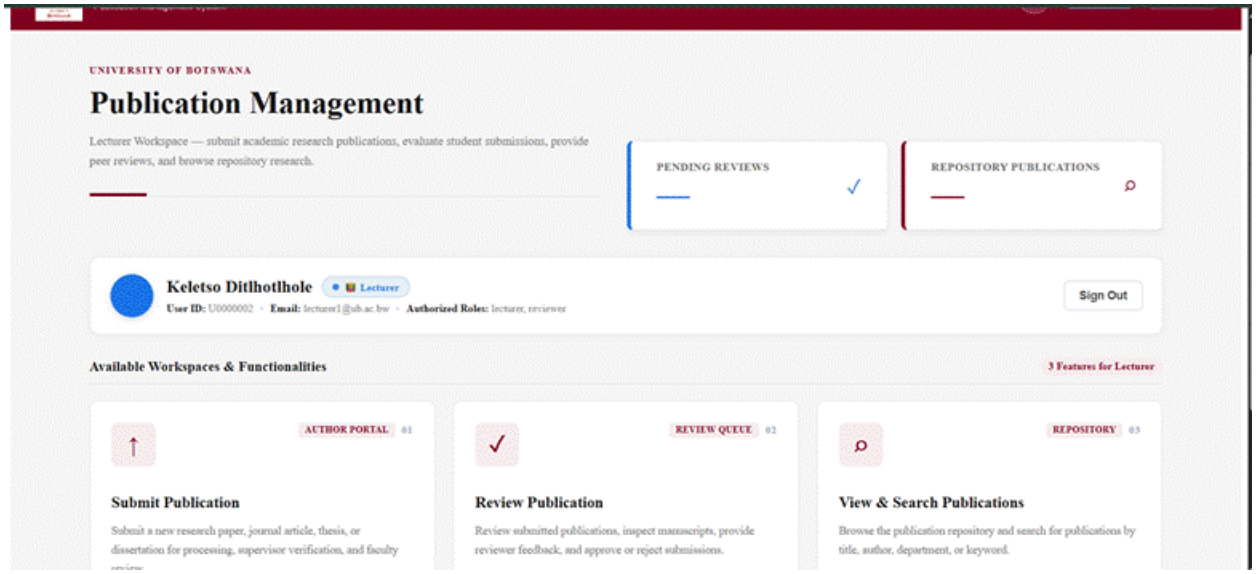
Detailed Format Summary Registry

8 total records analyzed

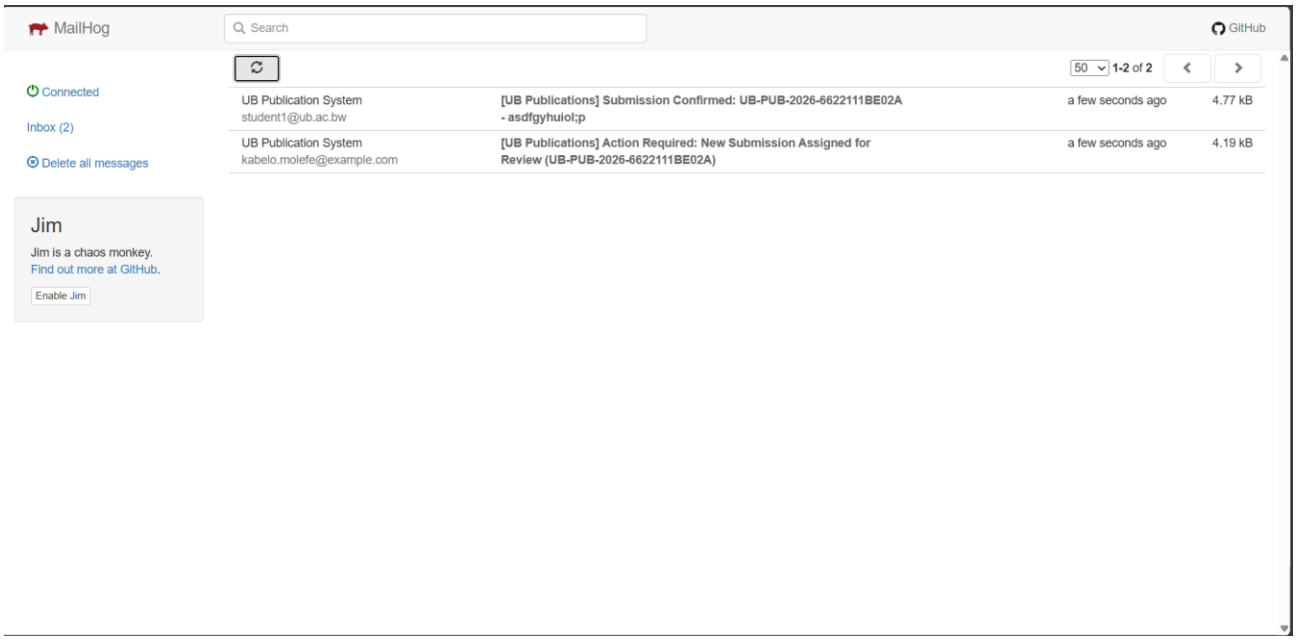
Format / Type	Approved Count	Share of Repository	Visual Share	Quick Action
Journal Article	4	50%		Filter View →
Conference Paper	2	25%		Filter View →
Thesis (Master's)	1	13%		Filter View →
Dissertation (PhD)	1	13%		Filter View →
Book / Technical Report	0	0%		Filter View →

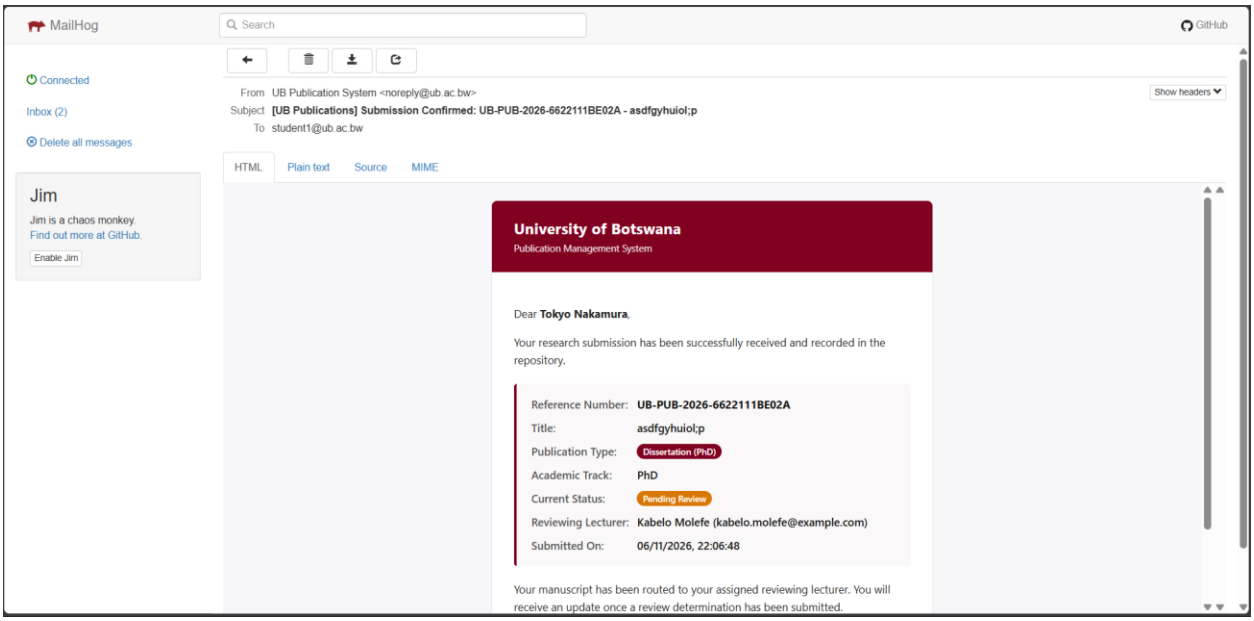


Lecturer Dashboard

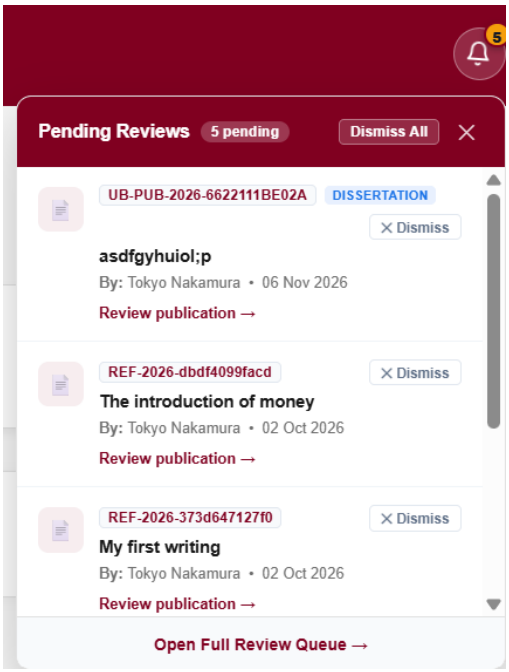


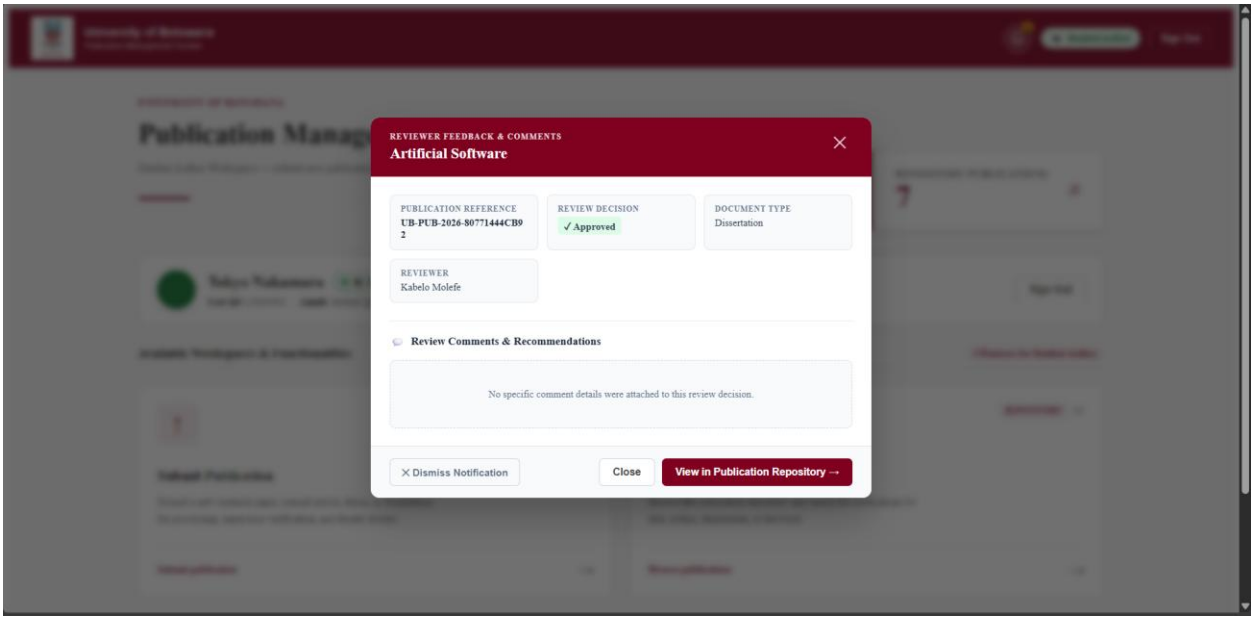
Notification Functionality - Email





Notification Functionality - Notification Bell In App




REVIEWER FEEDBACK & COMMENTS
Artificial Software
PUBLICATION REFERENCE
UB-PUB-2026-80771444CB9
2
REVIEW DECISION
✓ Approved
DOCUMENT TYPE
Dissertation
REVIEWER
Kabelo Molefe
Review Comments & Recommendations
No specific comment details were attached to this review decision.
✕ Dismiss Notification
Close
View in Publication Repository →

18. SYSTEM TESTING

18.1 Overview

We tested the UB Post-Graduate Research Publication Management System to check that it handles the whole publication lifecycle properly: submission, supervisor and peer review, repository search, analytics, and user profile management for theses, dissertations, journal articles and conference papers.

Scope and levels of testing

- **Unit and component checks:** We tested the frontend form validators (SubmitPublicationComponent and ReviewPublicationDetail), the Angular route guards (authGuard and roleGuard), and the backend middleware (authenticate, authorize and upload).
- **Integration and API testing:** We ran the REST endpoints in publication.routes.js and user.routes.js through Postman, using our Publications_API collection. This let us confirm that PostgreSQL transactions behave correctly, that Keycloak stays in sync through its Admin REST API, and that emails reach MailHog.
- **System and end-to-end testing:** We walked through full user journeys for each role: Guest, Student (Master's and PhD), Lecturer/Reviewer, and System Administrator.
- **Security and boundary testing:** We tried expired and tampered JWTs, self-review, non-PDF and oversized uploads, SQL injection through parameters, and CSV formula injection.

18.2 Test Environment and Test Accounts

Component	Address	What we used it for
Angular web app	http://localhost:4200	The user interface
Express backend API	http://localhost:3000	Business logic, role enforcement and file streaming
Keycloak	http://localhost:8180 (publications realm)	Issuing tokens, verifying them, and storing users
PostgreSQL 18	localhost:5432	Data storage and the status history audit trail

MailHog	smtp://localhost:1025 and http://localhost:8025	Catching the emails the system sends

We seeded these accounts before testing:

Role	User ID	Username	Level	Supervisor
Master's student	STU000001	student1	Master's	LEC000001
PhD student	STU000002	student2	PhD	LEC000001
Lecturer/Reviewer	LEC000001	reviewer1	Faculty	LEC000002 (peer lecturer)
System administrator	ADM000001	admin1	N/A	N/A
Guest	none	none	none	none

18.3 Test Cases

18.3.1 Authentication, sessions and role-based access

ID	Test	What we did	What we expected	Result
TC-AUTH-01	Valid login	With Keycloak and the backend running, we logged in on /login as student1.	Keycloak returns a JWT, /api/me adds the database ID (STU000001), roles and Master's level to the profile, and we land on /dashboard.	PASS
TC-AUTH-02	Wrong password	We entered a wrong password for student1.	Keycloak returns 401 invalid_grant and the page shows "Invalid username or password."	PASS
TC-AUTH-03	Keycloak down	We stopped the Keycloak container and tried to log in.	AuthService catches the status 0 error and shows "Unable to connect to authentication server. Please ensure Keycloak is running."	PASS
TC-AUTH-04	Deactivated account	We set enabled = false for STU000001 and called a protected endpoint with that user's token.	verifyToken sees the account is disabled and returns 403 with "Your account has been deactivated by an administrator."	PASS
TC-AUTH-05	Expired or tampered token	We sent an expired or altered JWT to /api/publications/review.	RS256 verification against the JWKS fails and the API returns 401 ("Token has expired" or "Invalid or expired token").	PASS

TC-AUTH-06	Auto-provisioning	We created a user directly in Keycloak (not in PostgreSQL), then logged in and called /api/me.	resolveDbUser generates the next STU/LEC/ADM ID, assigns a default supervisor to students, and inserts the user into PostgreSQL.	PASS
TC-AUTH-07	Route guards	While logged in as a student, we opened /review-publication and /manage-profiles, and called the review and admin APIs.	The Angular roleGuard sends us back to /dashboard, and the backend returns 403 "Insufficient permissions".	PASS
TC-AUTH-08	Dashboard cards per role	We logged in as a guest, student, lecturer and admin in turn.	Guest sees View & Search and Sign In. Student sees Submit Publication and View & Search. Lecturer sees Submit, Review Publication and View & Search. Admin sees only Manage Profiles and Generate Report.	PASS

18.3.2 Publication submission

ID	Test	What we did	What we expected	Result
TC-SUB-01	Valid Master's thesis	As STU000001, we submitted a complete form with a valid PDF under 100 MB.	201 Created. Rows are inserted into Publications (status Pending, reference UB-PUB-2026-XXXXXX), Publication Author, Publication File and Publication Status History, all in one database transaction.	PASS
TC-SUB-02	Master's student submits a dissertation	As STU000001, we posted a publication of type Dissertation.	400 Bad Request: "Dissertations are reserved for PhD students. Master's students must submit a Thesis."	PASS
TC-SUB-03	PhD student submits a thesis	As STU000002, we posted a publication of type Thesis.	400 Bad Request: "Theses are reserved for Master's students. PhD students must submit a Dissertation."	PASS
TC-SUB-04	Student submits a journal article	As STU000001, we posted a Journal Article.	403 Forbidden: "Only lecturers are able to submit Journal Articles and Conference Papers. Graduate students must submit a Thesis or Dissertation."	PASS
TC-SUB-05	Lecturer submits a thesis	As LEC000001, we posted a Thesis.	400 Bad Request: "Lecturers can only submit Journal Articles or Conference Papers."	PASS
TC-SUB-06	Peer reviewer assignment	As LEC000001, we submitted a valid journal article or conference paper.	submitPublication assigns a different lecturer as reviewer, so nobody reviews their own work.	PASS

TC-SUB-07	Duplicate title	We submitted “ test paper “ when “Test Paper” already existed.	409 Conflict: “A publication with this title already exists.” The temporary file is deleted.	PASS
TC-SUB-08	1,000-word abstract limit	We entered a 1,005-word abstract.	Both the frontend and the backend reject it and say it is over by 5 words.	PASS
TC-SUB-09	Title and author length	We used a title over 155 characters, then an author name over 50.	400 Bad Request: “Title (max 155 characters) or an author name (max 50 characters) is too long.”	PASS
TC-SUB-10	Non-PDF upload	We attached a .docx, a .png and a file with a spoofed extension.	The file filter returns 400: “Only PDF files are allowed.”	PASS
TC-SUB-11	PDF over 100 MB	We attached a 105 MB PDF.	Multer raises LIMIT_FILE_SIZE and the error handler in server.js returns “The PDF must not exceed 100 MB.”	PASS
TC-SUB-12	Rollback and cleanup	We forced a failure after the PDF had uploaded (for example, a duplicate title).	The transaction rolls back and publication.controller.js deletes the temporary PDF from backend/uploads.	PASS

18.3.3 Peer and supervisor review

ID	Test	What we did	What we expected	Result
TC-REV-01	Review queue	LEC000001 had their own paper plus student papers waiting. We called GET /api/publications/review?page =1&limit=10.	Only items with status submitted, pending or rejected and a submitter other than LEC000001 come back. Their own paper is left out.	PASS
TC-REV-02	Reviewing own paper	LEC000001 sent PATCH /api/publications/:ownId/status with Approved.	403 Forbidden: “You cannot review, approve, or reject a publication that you submitted yourself. Another lecturer must review it.”	PASS
TC-REV-03	Commenting on own paper	LEC000001 posted a comment on their own publication.	403 Forbidden: “You cannot add review comments to your own publication. Another lecturer must review it.”	PASS
TC-REV-04	Adding a comment	As reviewer on a student’s paper, we posted a comment with a title (150 characters or fewer) and details.	201 Created with id, title, details, createdAt and reviewerId, and the comment appears at the top of the list.	PASS

TC-REV-05	Comment validation	We submitted an empty comment, then a title longer than 150 characters.	400 “Comment title or details are required” for the first, and 400 “Comment title must not exceed 150 characters.” for the second.	PASS
TC-REV-06	Approve or reject	On a pending student paper with a comment, we clicked Approve, then Reject.	The status updates, a Publication Status History record is added and linked to the latest comment, and a decision email goes out.	PASS
TC-REV-07	PDF in review view	We opened /review-publication/:id and clicked View Manuscript.	The PDF streams inline and Angular shows it in the iframe through a Blob URL.	PASS
TC-REV-08	Sort, filter, paginate	With several items in the queue, we filtered by type and status, searched by keyword, and sorted by submitter and by oldest.	Sorting, filtering and pagination all work, and an out-of-range page jumps to the last page.	PASS

18.3.4 Repository search, visibility and analytics

ID	Test	What we did	What we expected	Result
TC-REP-01	Guest and student visibility	We browsed /search-publications as a guest and as a student, then requested a pending publication and its file directly by ID.	Only Approved publications show up, and direct requests for unapproved ones return 404.	PASS
TC-REP-02	Staff visibility	As a lecturer and as an admin, we called GET /api/publications and requested a pending publication by ID.	Staff see publications in every status.	PASS
TC-REP-03	Search and filters	We searched by title, author, reference code, keyword, type, year, subject area and date range.	Search is debounced by 300 ms, and the filters stay in sync with the URL and update the results.	PASS
TC-REP-04	Analytics and chart drill-down	With the analytics panel open, we switched between Approved and Filtered and clicked a slice of the donut chart.	The panel shows type distribution, academic track, a 2023 to 2026 timeline, the top 8 keywords and the multi-author rate. Clicking a type filters the list and scrolls to it.	PASS

TC-REP-05	Notification bell	After STU000001's publication was approved or rejected, we clicked the bell on /dashboard and opened the notification.	A feedback modal shows the status, the assigned lecturer and all reviewer comments. Notifications can be dismissed one by one or all at once, and this is saved in localStorage.	PASS

18.3.5 Administration, Keycloak sync and reports

ID	Test	What we did	What we expected	Result
TC-ADM-01	Create a user	As ADM000001 on /manage-profiles, we created a PhD student with supervisor LEC000001.	An ID like STU00000X is generated, the user is inserted into PostgreSQL, the user, roles and attributes are created in Keycloak, and keycloak/realm-export.json is updated.	PASS
TC-ADM-02	Duplicate ID or email	We tried to create a user with an existing ID or email.	409 Conflict: "A user with ID ... or email ... already exists."	PASS
TC-ADM-03	Two-way sync	We changed a user's name and role in Keycloak, added a user only in PostgreSQL, then clicked Sync with Keycloak.	The Keycloak changes come into PostgreSQL, and the PostgreSQL-only account is pushed into Keycloak.	PASS
TC-ADM-04	Admin deleting themselves	We sent DELETE /api/admin/users/ADM000001 as that admin.	400: "You cannot delete your own active administrator account."	PASS
TC-ADM-05	Deleting a user who has submissions	We tried to delete STU000001, who has submitted publications.	400: "User STU000001 has N submitted publication(s). Deactivate their account instead of deleting to preserve repository history."	PASS
TC-ADM-06	Reports and CSV injection	In the Generate Report modal, we downloaded a filtered CSV and a PDF, and used a title starting with =cmd.	The CSV (UTF-8 BOM) includes Summary, Type Breakdown, Reviewer Workload and Registry. The PDF is A4 landscape. Cells starting with =, +, - or @ get a leading ' so spreadsheets don't run them as formulas.	PASS

18.3.6 Email notifications (MailHog)

ID	Test	What we did	What we expected	Result
TC-EML-01	Submission confirmation	A student submitted a new thesis.	The student gets an HTML and plain-text email with the reference code, the track (Thesis (Master's)), the status Pending Review and the assigned lecturer.	PASS

TC-EML-02	Lecturer assignment	A student, then a lecturer, submitted a publication.	The assigned lecturer gets an action-required email.	PASS
TC-EML-03	Decision email and HTML escaping	A lecturer approved or rejected a publication, with feedback containing <script>.	The email shows a green (Approved) or red (Rejected) badge, and the feedback is escaped as <script>. If SMTP is offline, nothing breaks.	PASS

18.4 Summary of Results

Module	Total	Passed	Failed	What it covered
Authentication and access control	8	8	0	Login, token checks, deactivated users, auto-provisioning, route guards
Publication submission	12	12	0	Master's, PhD and lecturer rules, 1,000-word limit, 100 MB PDF limit, rollback
Peer and supervisor review	8	8	0	Self-review blocking, comments, status changes, inline PDF
Repository search and analytics	5	5	0	Visibility by role, filters synced to the URL, charts, notification bell
Admin and Keycloak sync	6	6	0	Two-way sync, delete protection, CSV/PDF reports, CSV injection defence
Email notifications	3	3	0	Confirmation, assignment, decision emails, HTML escaping
Total	**42**	**42**	**0**	**All functional, integration and security tests passed**

19. RECOMMENDATIONS FOR FUTURE IMPROVEMENTS

After testing, we noticed several things we would improve in the future.

Functional and workflow improvements

1. Editing and resubmitting rejected publications. Right now, when a student clicks Resubmit from the feedback modal, they land on a blank /submit-publication form. They have to retype the title, abstract, keywords, subject area and authors, and if they reuse the same title the system rejects it with 409 Conflict. We recommend keeping rejected publications in a "My Submissions" area. A Modify & Resubmit button would fill the

form with the saved details, let the student change anything and upload a new PDF, then send it through PUT /api/publications/:id/resubmit. That would set the same publication back to Pending and keep the full history in Publication Status History.

2. Longer login sessions and silent token refresh. We didn't set accessTokenLifespan in realm-export.json, so Keycloak uses its 5-minute default. If a student spends longer than that writing an abstract, or a lecturer spends longer reading a thesis before commenting, authInterceptor sees the token has expired, clears localStorage and signs them out, and their work is lost. We suggest two fixes: raise accessTokenLifespan to 3600 or 7200 seconds, with ssoSessionIdleTimeout at 7200s, and store the refresh_token in AuthService so authInterceptor can refresh the token (grant_type=refresh_token) while the user is still active.

3. Review queue filtered by supervisor. getReviewList currently returns every pending submission in the department except the lecturer's own. We would add an "Assigned to Me" toggle (pub.lecturer_id = req.user.id) so supervisors can quickly find their own supervisees.

Architecture, scalability and DevOps

- **Docker Compose.** docker-compose.yml only runs Keycloak and MailHog at the moment. We would add postgres, backend and frontend containers with named volumes so the data persists